

绿 皮 书

Agent Memory 智能体记忆全景研判

第一性原理 · 机制拆解 · 流派谱系 · 评测真伪 · 选型决策

定位：面向对 Agent 记忆感兴趣的读者与工程实践者的全景研判——回答"记忆问题本质上是什么、领域整体走到哪、哪些说法可信、该怎么选"。

证据底座：12 个开源记忆系统源码逐行实测 + 2025–2026 研究侧全景（Mem0 v2.0.20、Graphiti 0.30.2、Letta 0.16.8、LangMem 0.0.30、MemOS 2.0.33、HippoRAG2、Cognee、A-MEM 等）+ 2026 年 9 月公开评测与行业动态，全部结论标注可访问出处。

编者：老刘说NLP技术社区 · 刘焕勇 · <https://liuhuanyong.github.io>

版本：v3.1（增补企业上下文章节） · 2026 年 9 月 26 日

目录

目录

为什么在 2026 年还需要一本"绿皮书"

第一部分 · 根基

第 1 章 第一性原理：Agent 为什么必须记住

- 1.1 从公理出发：无状态的模型，有状态的任务
- 1.2 "上下文窗口够大"能不能终结记忆问题
- 1.3 三种解法谱系：同一个矛盾的三条出路
- 1.4 演进时间线：从论文到产品标配用了三年
- 1.5 记忆是产品竞争力的护城河吗
- 1.6 行业隐含假设清单：五条"没人写进文档但人人遵守"的前提

第 2 章 记忆的科学基础与分类学

- 2.1 认知科学的映射：借来的词汇表
- 2.2 工程师的六角色分类：比"三分法"更贴近实现
- 2.3 一个容易混淆的区分：Agent 档案 vs 用户档案
- 2.4 "好记忆"的评价维度：比"准不准"更完整的五维

第二部分 · 机制

第 3 章 记忆生命周期总架构

- 3.1 一个闭环，六道工序
- 3.2 关键非对称：写路径是异步的，读路径是同步的
- 3.3 分层：哪些东西放在哪里
- 3.4 多 Agent 共享记忆：三种拓扑

第 4 章 写入：抽取、编码与冲突消解

- 4.1 先看坐标系：抽取的三种范式与一个关键实证
- 4.2 Mem0：八阶段流水线与"只做加法"的转向
- 4.3 Graphiti：把"事实的时间性"编码进数据模型
- 4.4 LangMem：热路径与后台反思两条写姿势
- 4.5 CrewAI：写入管线 Flow 化与"合并四象限"
- 4.6 Cognee：写入与富化解耦的流水线
- 4.7 A-MEM：写入时演化（evolution-on-write）
- 4.8 OpenHands 与 Letta：文件即记忆与 Agent 自写
- 4.9 五种写策略的本质差异（汇总）

第 5 章 检索：从向量到图

- 5.1 纯向量检索的天花板
- 5.2 五个实测样本
- 5.3 重排器的分层：便宜信号在前，贵的在后
- 5.4 OpenHands 的提醒：检索有时可以没有
- 5.5 读检索数字之前：三个必须核对的事

第 6 章 遗忘、压缩与巩固

- 6.1 压缩的两个实测样本
- 6.2 巩固：从"记住"到"记得更好"
- 6.3 该忘什么、不该忘什么：遗忘的分界线
- 6.4 遗忘的合规面：被遗忘权进入工程清单

第三部分 · 流派

第 7 章 流派全景：六条技术路线

- 7.1 研究侧的第七条路线：记忆资源调度派（2025 年成形）
- 7.2 十二系统能力矩阵（源码实测归纳）
- 7.3 同构性：六条路线共享的底层机械

第 8 章 精讲 · Mem0：向量事实层派

- 8.1 它到底是什么
- 8.2 多租户命名空间（工程上最值得抄的部分）
- 8.3 检索、过滤与静默降级

- 8.4 双实现、遥测与 OSS/平台边界

第 9 章 精讲 · Zep/Graphiti：时序知识图谱派

- 9.1 Graphiti 是内核，Zep 是商业云
- 9.2 数据模型：五类节点、五类边、双时间轴
- 9.3 索引与检索的落地细节
- 9.4 工程画像与可运行示例
- 9.5 数字怎么看：自报基准与三方争议

第 10 章 精讲 · Letta 与 Generative Agents：自编辑与认知仿真

- 10.1 Letta（MemGPT）：把记忆当操作系统资源
- 10.2 Generative Agents：一切设计的认知科学母本

第 11 章 精讲 · HippoRAG / Cognee / A-MEM：三个不同方向的探索

- 11.1 HippoRAG2：神经-图派的严谨样本
- 11.2 Cognee：把"记忆化"做成可组合流水线
- 11.3 A-MEM：Zettelkasten 卡片盒的教训

第 12 章 产品级记忆：从 Dreaming 到 AGENTS.md

- 12.1 ChatGPT：Dreaming 与 "被管理的遗忘"
- 12.2 Anthropic：memory tool + 上下文编辑的开发者路线
- 12.3 其他平台的快照
- 12.4 Markdown 文件的胜利
- 12.5 Basic Memory：文件派与 MCP 的合流
- 12.6 编码 Agent 的"文件即记忆"谱系
- 12.7 治理面：消费级记忆正在被监管重塑

第三部分 · 延伸

第 13 章 延伸 · 企业上下文：组织级记忆与 Agent 记忆的会师

- 13.1 是啥：定义与边界
- 13.2 为啥现在爆发：三个事实
- 13.3 怎么做：一条被验证的工程路径
- 13.4 谁在做：三种代表形态
- 13.5 与 Agent 记忆的关联：同根、分层、双向流动
- 13.6 挑战：六个真实的坑
- 13.7 场景与建议

第四部分 · 研判

第 14 章 评测的真伪：基准通胀与独立复现

- 14.1 基准地图：五个主流基准各测什么
- 14.2 宣称 vs 复现：2026 年最重要的一张对照表
- 14.3 测量工具本身在漏：基准的两处结构性缺陷
- 14.4 自建评测的最低配置（可操作）
- 14.5 被"记忆"一词包装过头的三个宣称

第 15 章 工程选型：成本、边界与决策框架

- 15.1 成本结构：账单长在哪里
- 15.2 一个务实的决策框架
- 15.3 分阶段路线图：Day-1、Day-30、Day-90
- 15.4 失效模式清单（生产环境的 Day-2 问题）

第 16 章 开放问题与前瞻研判

- 16.1 八个未解的根本问题
- 16.2 阶段性研判（2026–2030）
- 16.3 三层读者各自能带走什么

附录

附录 A 术语表

附录 B 主要参考文献与数据来源

编者按

为什么在 2026 年还需要一本"绿皮书"

这本书选择了绿色——在出版惯例里，绿皮书（Green Paper）是**咨询性、研讨性文档**：它不只陈述事实，更要提出问题、给出研判、并诚实地标注每个判断的边界。这恰好是 Agent Memory 这个领域当下最缺的东西。

2025 到 2026 年，记忆从研究课题变成了产品基础设施：ChatGPT 上线 Dreaming 机制，Anthropic 发布 memory tool 并把跨会话记忆做进 Managed Agents，Google 推出 Memory Bank，四家托管记忆层在同一时期收敛发布。但热闹之下有三个不对称：其一，**厂商宣称与独立复现差距巨大**——同一基准上 94.4% 的宣称成绩，第三方复现只有 57.5%（第 14 章给出完整对照）；其二，**"记忆"一词同时指代至少七种机制截然不同的技术路线**，彼此的术语、评测口径与适用边界互不相通；其三，**绝大多数选型文章回避成本结构、失效模式与治理边界**——而这些恰是系统上线后最先撞上的东西。本书试图把这三个不对称摆到台面上。

全书 16 章 + 2 个附录，按"根基 → 机制 → 流派 → 延伸 → 研判"推进：第一部分（第 1-2 章）从"Transformer 无状态"这一公理推出记忆的根本矛盾，并给出认知科学基础上的分类学；第二部分（第 3-6 章）拆解写入、检索、遗忘与巩固的完整生命周期，结论落到源码行级；第三部分（第 7-12 章）纵览技术路线并精讲 12 个开源记忆系统，全部来自对开源仓库的逐行实测；**第 13 章为延伸章，讨论企业上下文——组织级的治理型上下文层与 Agent 个体记忆的会师**，这是 2026 年架构演进最活跃的方向之一，也回答"为什么 Agent 不了解我们公司"这一企业落地的高频问题；第四部分（第 14-16 章）处理评测的真伪、工程选型的决策框架与八个开放问题。每一章都遵循同一纪律：论据先行、结论加限定、反方观点与失效条件并置。

阅读约定：① 全书遵循「研判 | 边界与辩证」双栏结构——左栏给基于证据的阶段判断，右栏给反方观点、失效条件与待校验命题；② 术语先定义后使用，首次出现的术语有定义框，书末附录 A 收录完整术语表；③ 所有引用给出可访问的完整链接；④ 所有源码级事实来自对开源仓库的逐行实测（版本见正文），带行号引用可复核；⑤ 厂商口径的数字一律显式标注"自报/未经独立审计"，与独立复现结果分开呈现。

第一部分 · 根基

第 1 章 第一性原理：Agent 为什么必须记住

1.1 从公理出发：无状态的模型，有状态的任务

要理解 Agent 记忆，只需要从一条公理出发：**Transformer 是无状态函数**。每一次调用，模型只能看到本次输入的 token 序列，上一次调用中的任何内容——除非再次出现在输入里——对模型而言不存在。这条公理没有例外：不是"模型记性不好"，而是**模型在架构上没有记忆这个概念**。KV Cache 只是把"当前这次调用的状态"在生成期间缓存住，调用结束即弃——它加速的是单次推理，不是跨调用的记忆。

而真实任务是有状态的。把"有状态"拆到最小单元，是四类跨时间的信息：

信息类别	具体例子	时间跨度	如果丢失会怎样
用户事实	"对花生过敏""发票走 PDF""住在 B 城"（旧的"住 A 城"失效）	月-年	重复提问、犯错、体验崩坏
任务状态	上次跑到第几步、哪些方案已否决、PLAN.md 里的执行计划	小时-天	重做已完成的工作
协作约定	"这个仓库用 pnpm 不要 npm""别再生成单元测试"（用户纠正）	永久	反复违反同样的约定
组织知识	客户档案、决策记录、实体关系（"A 曾在 X 公司、后加入 Y"）	年	每次会话从零重建上下文

这四类信息的失效节奏完全不同——这是后续所有设计分歧的根源：用户事实会"被更新"（地址变了），任务状态会"被完成"（做完即失效），协作约定近乎永久，组织知识则慢慢陈旧。一个只用单一存储、单一衰减策略的系统，注定在某一类上出错。第 2 章的六角色分类、第 4 章的写策略、第 6 章的遗忘形态，都是围绕这张表的差异化展开。

于是产生了一个不可避免的根本矛盾——

根本矛盾：模型的状态窗口（上下文窗口）有限、易失、按 token 计费；任务需要的状态跨会话、长期、随时可能失效。Agent Memory 的全部工程，本质上都是在做一件事：决定哪些状态、以什么表示、在什么时机、花多少成本，进入或离开那个有限的窗口。

这个定义比常见的"给 LLM 加个向量库"要严格得多。按这个定义，记忆系统必须同时回答四个子问题，缺一不可：

子问题	含义	对应的失效模式	本书对应章节
抽取（Extraction）	从原始对话/行动流里提炼什么值得存	存了整段流水账，检索噪声淹没信号	第 4 章
表示（Representation）	以向量/文本/图/文件的什么形态存	向量分不清"曾经为真"与"现在为真"	第 7 章
检索（Retrieval）	新一轮里，哪些记忆该被换入窗口	语义相近但不相关；漏掉多跳关联	第 5 章
失效（Invalidation）	事实变化时旧记忆怎么办	用户改了地址，Agent 还用旧的	第 4、6 章

四分法与 2026 年行业共识一致；2025 年 12 月的大规模综述《Memory in the Age of AI Agents》进一步给出了"形式/功能/动态"三轴分类学与"平面检索型 / 显式结构型 / 策略管理型"三范式划分，参见：arxiv.org/html/2603.04740（该文对此综述有转述）。

1.2 "上下文窗口够大"能不能终结记忆问题

常见的反驳是：窗口在变大（百万 token 已可用），把所有历史塞进去不就行了。四条证据说明这条路目前走不通：

- 成本与延迟随窗口线性增长：每轮都携带全部历史，token 计费与首 token 延迟随之上升。Anthropic 官方数据显示，配合记忆工具与上下文编辑，100 轮网页搜索评测中 token 消耗降低 84%——反证了"全量进窗口"在工程上不可接受。出处：o-mega.ai/articles/ai-agent-memory-in-2026-architectures-tools（引 Anthropic 官方数据）。
- 长窗口下检索精度下降（Context Rot）：随上下文变长，模型对关键信息的利用效率下降。BEAM（Beyond a Million Tokens）基准在 128K 到 10M token 尺度上、以 2000 道题测试，公开最高分约 64——远低于短上下文表现；该基准的明确设计目标就是让"全量塞上下文"拿不到虚高分。出处：clawdbot2.in/how-does-ai-agent-memory-work-2026。
- 窗口解决"装不下"，不解决"知道什么是真的"：窗口里同时躺着"用户住在 A 城"和"用户搬到 B 城"两条信息时，模型没有可靠的机制判断哪条现在有效。这正是时序失效问题，窗口本身不提供解。
- 全量进窗口的"朴素上界"实测很低：在 LoCoMo 的 GPT-4.1-mini 设置下，全量长上下文直接作答的 F1 只有 18.70，而带记忆压缩的 SimpleMem 拿到 43.24、Mem0 拿到 34.20——把所有信息堆给模型，效果反而最差。出处：SimpleMem 论文（2026.01），数据转引自 o-mega.ai/articles/self-improving-ai-agents-the-2026-guide。这一条最反直觉也最有说服力：问题不是"装不下"，而是"装下也读不好"。

补一笔具体的账（数量级演示，非精确测算）：设一个助手级 Agent 每轮平均产生约 0.4K token 的新对话、50 条抽取记忆约 2K token。走"全量历史"路线时，第 100 轮的输入约为 100 × 0.4K（全部前文）+ 新消息 ≈ 40K+ token/轮，且随轮数线性增长；走"记忆层"路线时输入稳定在 10–12K token/轮。在按输入 token 计费的模型下，两条路线的成本差在长会话里是数倍到数十倍，且前者还叠加了长上下文精度损失。这是外部记忆层存在的经济学理由。

再给一个厂商侧的量级参考（注意是厂商口径）：一个典型客服 Agent 每次交互约消耗 3150 输入 token + 400 输出 token，按 1 万日活、人均 5 次交互估算，月度 token 成本在 2000–5000 美元区间，而引入记忆压缩类方案可压掉一半以上；SimpleMem 则宣称在保持更高 F1 的同时把推理期 token 消耗最多降低 30 倍。出处：youmind.com/blog/memos-ai-agent-long-term-memory-system；SimpleMem 数据同上。两处数字都未经独立审计，看量级即可。

研判

- 在现有证据下，"外部记忆 + 小窗口"在成本与精度上仍优于"全量塞窗口"，且这一差距在长周期任务上会拉大。
- 记忆系统的核心价值不是"能存"，而是**策展（curation）**：决定什么进窗口。全量长上下文 F1 反而最低的实测说明：策展失败比容量不足更伤。

边界与辩证

- 百万窗口 + prompt caching 正在压缩外部记忆的成本优势（稳定前缀可缓存复用），边界在向“窗口内”移动，需每年重估。
- “长窗口精度下降”的证据主要来自 BEAM 等基准，真实工作负载的衰减幅度未必相同；“全量上下文 F1=18.70”也仅是单一模型（GPT-4.1-mini）单一数据集（LoCoMo）上的观察，换更强模型与更高质量转写，差距会缩小。
- 对超短周期任务（单会话内），外部记忆纯属过度设计。
- 上面的成本账是数量级演示：实际数字取决于模型定价、缓存命中率与会话长度分布；引用的厂商口径数字未独立审计。

1.3 三种解法谱系：同一个矛盾的三条出路

把 2023–2026 年的所有方案放回根本矛盾上，只出现三种解法，对应记忆的三个“放置位置”：

解法	思路	代表	一句话评价
① 窗口内管理	不建外部存储，对窗口本身做压缩/裁剪/分页	滚动摘要（LangMem SummarizationNode）、Anthropic context editing、OpenHands condenser、Claude Code compaction、Mastra Observational Memory（Observer/Reflector 双后台代理把历史压成“带日期的观察日志”常驻窗口）	便宜、无基础设施，但压缩通常是有损的
② 窗口外系统	建外部存储，按需换入换出	Mem0、Zep/Graphiti、Letta archival、Cognee、HippoRAG2、MemoryOS（STM/MTM/LPM 三层）、MemTree（动态树）	本领域主战场，能力上限最高、工程成本也最高
③ 架构内改造	把记忆做进模型权重或激活状态	MemOS 的 para_mem（LoRA 参数记忆，“即将推出”）、KV Cache 激活记忆（act_mem）	方向诱人但 2026 年仍处研究期，无成熟生产方案

③ 的定位依据 MemOS 2.0.33 源码逐行实测：MemoryFactory.backend_to_class 枚举含 kv_cache/vllm_kv_cache/lora 后端，但 README 标注参数记忆“即将推出”、Dream 为 beta。论文：arxiv.org/abs/2507.03724。Mastra Observational Memory 的定位转述自 arxiv.org/html/2603.04740。

三条出路不是互斥的，严肃系统通常是三者组合——这正是 Anthropic 官方 context engineering 指南给出的三技巧（compaction / structured note-taking / sub-agent architectures）与四大托管记忆层收敛架构背后的共同逻辑。2025 年 12 月综述的三范式划分（平面检索型 / 显式结构型 / 策略管理型）本质上是给③的内部再分层：平面型最简单但组织能力弱，结构型（图/树/分层）组织更强但模式僵化，策略管理型（由 LLM/策略决定读写时机）最灵活也最难做对。

1.4 演进时间线：从论文到产品标配用了三年

时间	事件	意义
2023.04	斯坦福 Generative Agents（Smallville）发布， arXiv:2304.03442	记忆流+三因子检索+反思，认知闭环母本
2023.10	MemGPT 论文， arXiv:2310.08560	"LLM 即操作系统"隐喻确立，上下文分页范式
2024.02	ChatGPT 上线 memory（初版仅显式 saved memories）	记忆首次成为消费级产品功能
2024.09	MemGPT 演进为 Letta 框架	自编辑记忆从论文走向 Agent 服务器
2025.01	Zep 时序知识图谱论文， arXiv:2501.13956	双时态事实建模成为独立流派
2025.02	HippoRAG 2 (arXiv:2502.14802)、A-MEM (arXiv:2502.12110)	神经-图派与 agentic memory 派成型
2025.05	MemTensor 发布 MemOS， arXiv:2507.03724 （7 月）	"记忆操作系统"叙事出现：记忆成为可调度资源
2025.10	Mem0 完成 2400 万美元 A 轮；API 调用量从 2025Q1 的 3500 万次涨到 Q3 的 1.86 亿次	记忆层第一次出现规模化的商业验证信号
2025.12	综述《Memory in the Age of AI Agents》发布；MemOS v2.0 "Stardust"（多模态记忆、工具记忆、Redis Streams 调度）；EMNLP 2025 Oral 收录 MemoryOS（北邮）	学界开始系统化整理；分层存储研究有了可复现成果（LoCoMo 上 +48.36% F1）

2026.01	SimpleMem 发布（结构化压缩+意图感知检索规划）；ICLR 2026 设 MemAgents workshop	效率路线（更高 F1、更低 token）成为新竞争轴
2025.09	Anthropic memory tool + context editing 同日发布	开发者级"文件即记忆"原语标准化
2026.04–06	Mem0 四月算法更新；OpenAI Dreaming 上线（2026-06-04）	厂商竞赛白热化，基准数字开始通胀
2026 年中	Weaviate Engram、Cloudflare Agent Memory、Microsoft Foundry Memory、Mem0 平台七周内先后发布	记忆成为云厂商标配基础件，架构收敛
2026.08	OpenAI 弃维 Assistants API（2026-08-26）；EU AI Act 透明度要求全面生效	状态管理从模型 API 层剥离；记忆治理进入监管视野

时间线综合 Taskade《The History of Agent Memory》taskade.com/blog/agent-memory-history、o-mega.ai 综述 o-mega.ai/articles/self-improving-ai-agents-the-2026-guide 与 youmind.com/blog/memos-ai-agent-long-term-memory-system；论文条目以 arXiv 号为准。Mem0 融资与调用量数据为厂商披露，未经独立审计。

这条时间线里藏着一个结构性判断：**2023–2024 年是"认知隐喻期"（Smallville、MemGPT 定范式），2025 年是"工程化期"（各流派成型、综述出现），2026 年是"基础设施化期"（云厂商入场、监管介入、基准通胀）**。每一期的主导问题不同——第一期问"记忆该长什么样"，第二期问"怎么实现"，第三期问"谁的可信、怎么治理"。本书的章节安排（根基→机制→流派→研判）正是沿这三期展开的。

1.5 记忆是产品竞争力的护城河吗

2026 年出现了一个值得注意的战略信号：ChatGPT、Claude、Gemini 三家全部开放了**记忆导入/导出**（如 Google 2026 年 3 月支持从 ChatGPT/Claude 迁移最多 5GB 聊天历史与记忆；Claude 支持导入 ChatGPT/Gemini/Grok 的记忆并与本地记忆合并而非覆盖）。当切换助手不再丢失积累的上下文，竞争回到模型质量与生态本身——这暗示厂商自己也判断：记忆数据本身不构成锁定，**记忆机制的质量才是竞争力**。出处：chatforest.com/guides/ai-agent-memory-patterns。

研判

- "记忆可迁移"对用户是利好，对第三方记忆层供应商是双刃剑：可迁移性削弱数据锁定，但也让"记忆质量"成为唯一比拼项。
- 需要区分"聊天历史可迁移"（已实现）与"语义记忆/图结构可迁移"（无标准格式，仍不可迁移，见第 16 章开放问题）。

边界与辩证

- 导入导出的互通仅限三大消费平台之间，开发者侧的托管记忆服务（Mem0/Zep/Letta）不在其列。
- "不构成锁定"是战略推断而非承诺：定价与 API 存续周期（Assistants API 前车之鉴）仍是事实上的锁定手段。

1.6 行业隐含假设清单：五条"没人写进文档但人人遵守"的前提

2026 年一篇提出"宪制记忆架构"的论文（[arXiv:2603.04740](https://arxiv.org/abs/2603.04740)）做了一件有价值的事：把当前所有记忆系统共享、却很少被明说的隐含假设列了出来。对照源码逐行实测，这些假设在工程层确实普遍成立：

隐含假设	工程上的实际表现（实测依据）	什么时候会出事
记忆属于用户或会话，不属于 Agent 本身	Mem0 三键（user/agent/run）、CrewAI scope 路径全部以"用户/会话"为一级归属；没有系统给 Agent 自身建"个人记忆"	Agent 需要跨用户积累经验（如"这类问题通常这样解"）时无处安放
Agent 只有单一生命周期	没有系统建模"前代实例"概念；Letta 的 agent 可以复制，但没有继承优先级语义	实例因故障/升级重建后，无法声明"我继承谁的哪些记忆"
治理由外部系统负责	谁有权写哪层记忆、错误记忆如何申诉，全部推给应用层；框架层没有 RBAC/ACL	多租户、合规场景（EU AI Act 2026.08 生效后这不再是理论问题）
遗忘等于删除	各系统的遗忘实现都是 TTL/删除/压缩，没有"存在但不被唤起"的中间态	被遗忘的记忆本可用于审计或恢复误删（参见第 6 章视图级遗忘的例外）
安全是数据保护问题	各系统只做传输/存储加密与 scope 隔离；把"污染 Agent 身份的记忆注入"当普通数据问题	提示注入把恶意事实写进记忆（第 15 章失效模式之一）

这张表的价值不在"正确", 而在**定位盲区**: 五条假设在"Agent 是短期工具"的场景下全部成立, 一旦 Agent 变成长期存续、承载身份的实体, 每一条都要重新审视。本书第 16 章的开放问题里有两条直接源于此。

第一部分 · 根基

第 2 章 记忆的科学基础与分类学

2.1 认知科学的映射：借来的词汇表

Agent Memory 领域的术语几乎全部借自认知科学。借用要诚实：**这是类比，不是同构**——大模型没有海马体, "遗忘曲线"只是衰减函数的隐喻。但类比提供了分类骨架：

认知科学概念	定义	Agent 系统对应物	实测实例
工作记忆	当下正在加工的少量信息	上下文窗口内的对话缓冲	最近 N 轮消息；Mem0 仅保留最近 10 条原文（storage.py:282-291 实测）
情景记忆（episodic）	带时间地点的事件记录	带时间戳的会话日志	Letta 的 recall memory（message 表）；MemOS 的 SourceMessage 溯源链
语义记忆（semantic）	去情境化的事实知识	抽取后的事实/知识库	Mem0 抽取的 "user prefers PDF invoices"
程序性记忆（procedural）	怎么做事的技能	Agent 的工作方式/约定	CLAUDE.md；Mem0 的 procedural_memory（main.py:817 实测，memory_type 仅允许此值）

三分法（episodic/semantic/procedural）已成为 2025–2026 年记忆文献的默认词汇表，参见繁体中文综述：signals.tw/articles/what-is-agent-memory/；2025 年 12 月综述在此基础上加了"形式/功能/动态"三轴，把分类从名词升级为可比较的坐标系（转述见 [arXiv:2603.04740](https://arxiv.org/abs/2603.04740)）。

认知分类学的工程补丁：参数记忆

认知科学没有、但工程上必须补的第四形态是**参数记忆（parametric memory）**：把经验直接写进模型权重（微调/LoRA）或激活状态（KV Cache）。MemOS 的 MemCube 抽象把它与明文记忆、激活记忆统一成一个可调度的资源单元（标注来源、重要性、使用历史，支持相互迁移）。这提醒我们：认知类比只覆盖了"窗口外文本"这一种形态，而工程系统正在试图统一三种形态。**一个实用推论**：稳定且高频复用的约定（如"本仓库永远用 pnpm"）适合降级为参数记忆或系统提示常量，而不是每次检索——检索是有成本的，越稳定的知识越不该走检索路径。

一个被直接借走的神经科学机制：海马体索引理论

HippoRAG 的名字直接来自**海马体索引（Hippocampal Indexing）理论**：人回想时，由少量线索（cue）经海马体快速扩散激活相关皮层记忆存储。HippoRAG2 把它机械化为：查询作为线索 → 图种子 → Personalized PageRank 扩散 → 激活篇章节点（HippoRAG.py:693 实测）。这是本领域少数把神经科学机制**算法化**而非只做隐喻的工作——但也要诚实：扩散激活只是借了形状，图上的 PPR 与生物扩散激活并无参数上的对应。

2.2 工程师的六角色分类：比"三分法"更贴近实现

认知科学三分法落进工程实现后，实际出现六种功能角色（对五个主流产品的调研归纳）：

角色	装什么	持久性	典型实现
工作记忆	当前会话上下文	易失	窗口内聊天历史
Agent 档案	该 Agent 的持久"性格/约定"	持久、按 Agent 分	CLAUDE.md、.cursorrules
用户档案	用户偏好/习惯/个人信息	持久、跨 Agent	ChatGPT 的 memory
情景记忆	按时间排序的交互日志	持久、带时序索引	JSONL 会话日志

语义记忆	可检索知识库	持久、带向量索引	RAG 向量库
日期锚定记忆	会过期的临时事实	带 TTL	"休假到 3 月 15 日"

出处：CrabTalk《How AI agents remember: a survey of persistent memory》crabtalk.ai/blog/persistent-agent-memory-research。该调研还指出：日期锚定记忆是最欠缺的一层——多数产品把临时事实和永久事实存成一样，导致它们永不过期。对照实测：Graphiti 的 `valid_at/invalid_at` 与 Basic Memory 的 `temporal.py`（TimeKind 含 `effective/valid/occurred/due/mentioned` 五种时间语义）是少数认真做了日期锚定的系统。

六个角色不是并列的——它们之间有**升格链条**。北邮 MemoryOS（EMNLP 2025 Oral）给出了目前最完整的显式实现：短期记忆（STM）用固定长度 FIFO 队列存近期对话页；队列满时最旧页按主题聚合进中期记忆（MTM）的 segment（聚合指标是语义+关键词相似度）；MTM 条目按**热度评分**（访问次数 + 对话页数 + 时间衰减）竞争升入长期个人记忆（LPM），低热度者优先驱逐。出处：o-mega.ai/articles/self-improving-ai-agents-the-2026-guide。这对应人类记忆的"复述→巩固"过程：一条信息被访问得越频繁，越值得升入更高层。**给你的可操作建议**：即便不用 MemoryOS，也应在自己的记忆表里加 `access_count` 和 `last_accessed` 两列——它们是后续一切升格/降格/驱逐决策的数据基础，事后补极为困难。

2.3 一个容易混淆的区分：Agent 档案 vs 用户档案

产品实现里最常被合并、也最不该合并的是前两者：**Agent 档案按 Agent 实例分**（这个项目怎么构建、那位用户偏好什么代码风格），**用户档案跨 Agent**（用户的时区、称呼、沟通风格）。Claude Code 的 CLAUDE.md 是 Agent 档案，ChatGPT 的 memory 是用户档案——把两者塞进同一个存储并共用同一套检索过滤，是新人最常见的建模错误。三个系统的实测处理方式：

- **Mem0**： `user_id/agent_id/run_id` 三键任一必填，写入与检索强制走命名空间过滤（`main.py`:817-851、1379 实测）；
- **CrewAI 1.15.21**：统一 `Memory` + 分层 `scope` 路径，如 `/crew/research-crew/agent/researcher/decisions` ——把"组织/团队/Agent/主题"编码进路径而非分库（`unified_memory.py` 实测）；
- **LangMem**： `namespace tuple` 模板（`"memories", "{langgraph_user_id}"`），运行时由 `configurable` 渲染占位符（`utils.NamespaceTemplate` 实测）。

三种方案殊途同归：**scope 是检索过滤的强制边界，不是可选标签**。判断一个记忆系统是否严肃，第一个检查项就是：检索 API 能否拒绝无 `scope` 的查询。Mem0 在 `main.py`:1379 直接校验三键必填——这就是"强制边界"的样子。

2.4 "好记忆"的评价维度：比"准不准"更完整的五维

基准只测"召回准不准"，但工程上一条记忆的价值至少有五个维度：

维度	问题	实现载体	评测代理
正确性	存的当初就抽对了 吗	抽取质量、few-shot、结构化校验	LoCoMo 单跳题；MemBench 的 fact 类
新鲜度	现在还成立吗	双时态（Graphiti）、TTL、衰减函数	LongMemEval 的知识更新类；Personamem 的"确认最新偏好"类
覆盖度	该记的都记了吗	抽取触发策略（热路径 vs 后台）、防抖窗口	LoCoMo 开放域题；MemBench 的 post-processing 类
关联度	散落各处的信息能 连起来吗	图结构、Zettelkasten 链接、同事事实同步更新	LoCoMo 多跳题；Personamem 的"偏好演化追踪"类
可溯源性/可 审计性	能指回证据、人能 纠错吗	Generative Agents 的 filling、Cognee 的 DataPoint 溯源、MemOS 的 sources 链、git diff	——目前没有任何主流基准测它

Personamem 的七类评测任务（召回事实/确认最新偏好/追踪偏好演化/回溯偏好成因/新建议/偏好对齐推荐/新场景泛化）出自 [arXiv:2602.14038](https://arxiv.org/abs/2602.14038) 附录——它是把"五维"拆成可操作题目的最好现成模板。

用这五维去量，很多营销叙事会现形：一个系统可以召回很准（第 1 维）但完全没有新鲜度机制（第 2 维），或精度好看但每条记忆都是黑盒（第 5 维为零）。特别注意第五维：**2026 年所有主流基准都没有"可溯源性"题目**——这是基准设计与工程需求的脱节处，也是第 14 章"自建 harness"要补的第一件事。

另一条来自大规模横评的结构性发现值得单独立牌：**保留原始消息的方法优于纯图三元组抽取（Mem0 优于 Mem0⁹）**——结构化抽取天然丢语义，信息完整性是记忆管理的第一基础；而**关联能力决定多跳上限**：缺乏关联操作的 MemoryBank、MemGPT 在多会话任务上表

现垫底，Mem0 靠"相似记忆同步更新"实现隐式关联后显著提升；**时间推理则强依赖模型能力**：模型从 7B 升到 72B，时间推理任务性能翻倍，说明现有系统普遍没有专门的时空处理组件，把活儿推给了底座模型。出处：[arXiv:2504.19413](#)、[arXiv:2505.22101](#)（横评转述）。

研判

- 工程上"六角色"比认知科学"三分法"更有操作性：它直接对应存储分片（namespace/scope）设计。
- 无论用哪个框架，**先画清 scope 边界**（user/agent/run/org）再选后端，顺序不能反。
- 五维评价里"可溯源性"被普遍低估：没有 provenance 的记忆在出错时无法定位根因，debugging 成本最高；而基准不测它，意味着选型时必须自测。
- "原始消息保底 + 结构索引加速"是横评给出的最优组合方向：纯抽取或纯图结构都有系统性损失。

边界与辩证

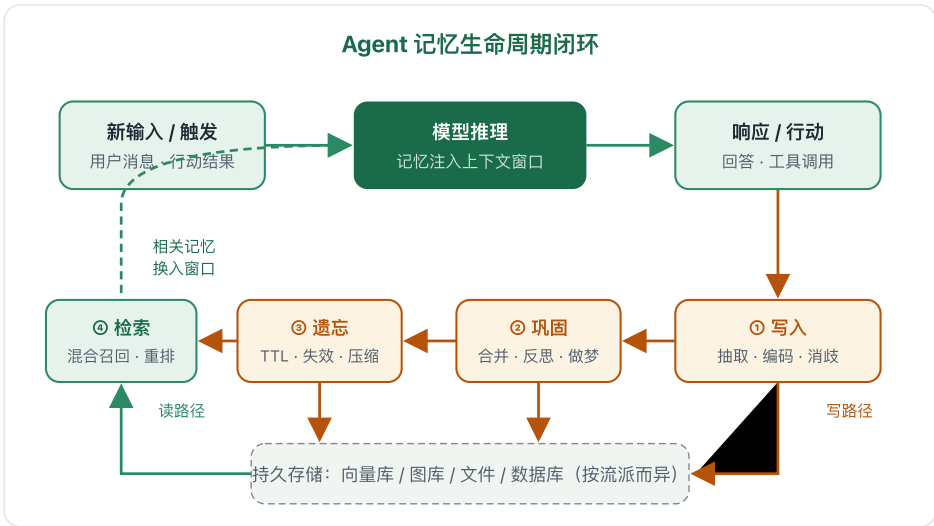
- 六角色边界在实践中互相渗透：Agent 档案里常混入用户偏好，框架层没有强制隔离。
- Mem0 用三元键、CrewAI 用路径、LangMem 用 tuple——同一概念在不同系统里粒度不同，不能直接互译。
- 认知类比有失效边界：人类遗忘是自适应的，工程衰减是静态公式；把两者等同会误导参数调优。
- 横评结论（原文优于图式、7B→72B 翻倍）来自特定基准与模型组合，比例会随模型代际变化，但方向性判断（信息完整性优先）相对稳健。

第二部分 · 机制

第 3 章 记忆生命周期总架构

3.1 一个闭环，六道工序

剥开各家系统的命名差异，所有严肃的记忆系统都在跑同一个闭环。这个闭环的两端都接在 Agent 主循环上——**读在推理前，写在推理后**：



图：Agent 记忆生命周期。橙色为写路径（写入→巩固→遗忘），绿色为读路径（检索→换入窗口）。该结构与 Taskade 对行业闭环的综述一致（[taskade.com/blog/agent-memory-history](#)），工序划分依据源码逐行实测归纳。

这个闭环还有一个更形式化的版本：2025 年综述把它拆成四组件流水线——**信息抽取 → 记忆管理 → 记忆存储 → 信息检索 → 生成回复**，并给每个组件列出了实现范式清单（抽取三种：直接归档/摘要式/图式；管理五操作：关联/整合/跨层转换/更新/过滤；存储两种组织：扁平/分层；检索四种：词汇/向量/结构/LLM 辅助）。出处：[arXiv:2504.19413](#)。本书第 4–6 章按生命周期展开，这两个坐标系可以互相映射：抽取+编码∈写入，管理五操作分布在写入与巩固，检索四范式∈第 5 章。

3.2 关键非对称：写路径是异步的，读路径是同步的

2026 年发布的四家托管记忆服务（Weaviate Engram、Cloudflare Agent Memory、Microsoft Foundry Memory、Mem0 平台版）在七周内收敛到了同一架构：**异步抽取-写入 + 检索优先召回，拒绝把东西塞满上下文窗口**。出处：dev.to/saaswithalex/ai-agent-memory-tiers-explained-what-actually-matters-1a82。

这个收敛不是巧合，而是延迟约束的必然：**读路径在同步请求链上**，用户每轮都在等它，所以检索必须快（Zep 宣称 p50 104ms）；**写路径可以异步**——写慢一点没关系，但不许拖住响应。四个系统的实测实现：

系统	异步写实现	一致性保障
LangMem	ReflectionExecutor 后台 daemon 线程 + after_seconds 防抖 (reflection.py 实测)	同 thread_id 新任务取消旧任务，避免重复抽取
MemOS	记忆增删改做成消息队列任务（RabbitMQ/Redis），SchedulerDispatcher 按 label 路由（mem_scheduler 实测）	Prometheus 指标 + 状态追踪；v2.0 改用 Redis Streams 做调度
CrewAI	remember_many 走单 worker 线程池串行化（unified_memory.py 实测）	读屏障 ：recall() 前强制 drain_writes()，读必见已提交写
Basic Memory	DB-first 写入 + 文件异步物化（note_content_materialization）	file_write_status 状态机；lifespan 关闭时排空队列，"已接受的写不丢"

读屏障（read barrier）是这组实现里最值得记住的工程细节：异步写换来了响应速度，代价是"写完立刻搜可能搜不到"，各系统用不同姿势补这个洞——CrewAI 靠 drain、Letta 写归档后强制重建系统提示、Mem0 的 messages 表同步写。**可操作建议**：自建记忆层时，把这个不一致窗口显式化——给每条写操作一个状态（pending/committed/failed），读侧要么阻塞到 commit，要么在结果里标注"含未巩固写入"。最糟的做法是假装一致：用户刚说完的事 Agent 说不知道，比没有记忆更伤信任。

反例同样有教育意义：MemGPT/Generative Agents 走的是**同步写**——MemGPT 的记忆写是 Agent 主动 function call，发生在对话流内；Generative Agents 的记忆流写入在每次"行动/对话"结束时同步 append。同步写简单、一致性强，代价是每次写入都在关键路径上加延迟。选择依据是交互节奏：聊天型（写频低、单条小）同步可接受；高频工具调用型（每轮产生大量中间结果）必须异步。

3.3 分层：哪些东西放在哪里

结合源码逐行实测，一个典型生产栈的分层如下（从快到慢、从易失到持久）：

层	承载内容	典型后端	源码实例
窗口内 (core)	始终可见的核心块	系统提示的一部分	Letta 的 persona/human block (letta/schemas/memory.py 的 compile() 渲染)；块字数上限实测 CORE_MEMORY_BLOCK_CHAR_LIMIT=100000
会话缓存	最近消息原文	Redis / SQLite	Mem0 的 messages 表仅留最近 10 条 (storage.py:282-291)
情景层	全量对话日志	Postgres + FTS	Letta 的 recall memory (message 表 + 混合检索)
语义层	抽取事实/知识	向量库 (±图库)	Mem0 的 Qdrant collection；Graphiti 的 Neo4j
归档层	冷数据	对象存储/归档表	Letta 的 archival memory (passage + 向量，可双写 Turbopuffer)

OpenHands 给出了这个分层的"文件系统变体"（TS 类型镜像实测）：跨会话长期记忆放仓库内 .openhands/memory/ 笔记目录（agent_context.load_memory 设置项，默认 false）、会话级计划落盘 PLAN.md 、会话内任务清单在内存（TaskTrackerObservation 回摄）——三层各对应一个时间尺度，全部是"文件即记忆"的形态。

研究侧则给出两个更激进的分层形态，各有可借鉴之处：**MemoryOS 的操作系统式三层**（STM 队列→MTM 主题段→LPM 画像，迁移规则完全显式：FIFO 下移、热度上移）适合需要可解释迁移策略的场景；**MemTree 的动态树**（节点含聚合文本+嵌入+抽象层级，新信息到来时按语义相似度决定合并/新建/分支）适合多粒度检索——顶层粗筛、叶层细查，横评显示树结构在 LongMemEval/LoCoMo 上表现居前但 token 开销也居前。出处：[arXiv:2505.22101](https://arxiv.org/abs/2505.22101) (MemTree)、o-mega.ai (MemoryOS)。**取舍要点**：层级越丰富，单次检索越准，但索引维护成本（树的深度、图的一致性）随更新频率上升——横评明确指出 MemTree 自顶向下插入成本随树深增长、Zep 图去重成本随图规模攀升，"频繁更新场景需权衡索引精度与维护开销"。

3.4 多 Agent 共享记忆：三种拓扑

记忆一旦从单 Agent 扩展到多 Agent，出现三种拓扑（均有源码实测依据）：

- **层级共享（scope 树）**：CrewAI 的路径式 scope——Crew 级记忆挂在 `/crew/{name}`，每个 Agent 的记忆挂在 `/crew/{name}/agent/{role}`，共享层与私有层由路径前缀天然隔离（crew.py 的 `create_crew_memory` 实测）；
- **消息传递（无共享存储）**：Letta 的 `send_message_to_agent*` 工具族（multi_agent.py 实测）——Agent 之间不共享存储，靠显式通信交换记忆，隔离最强、带宽最低；
- **隔离副本**：A-MEM 的 `CopiedChromaRetriever`（retrievers.py 实测）——为每个 Agent 克隆一份记忆库，读一致、写分裂。

选型口诀：**共享读、隔离写用 scope 树；强隔离用消息传递；只读镜像用副本**。三种拓扑的迁移成本差异巨大：scope 树只是路径约定，随时可改；消息传递需要重设计 Agent 间协议；副本分裂后几乎不可合并。多 Agent 记忆还有一个所有框架都没解决的前置问题——**写权限**（谁能写谁的记忆、写错了怎么撤销）：当前框架层完全没有权限模型，只能靠应用层在写入 API 外面包一层鉴权。这是第 1.6 节“治理缺失”假设在多 Agent 场景的具体化。

研判

- 分层是共识，但**层数不是越多越好**：每加一层就多一套检索配置、失效策略与调试面。
- 闭环里最被低估的是“巩固”（consolidation）——它是记忆质量随时间提升的唯一来源。
- 多 Agent 场景下 scope 树是当前最实用的共享拓扑，消息传递适合强隔离需求。
- 读写非对称（异步写+同步读）是延迟约束下的必然收敛，自建系统应默认照此设计，并把不一致窗口显式化。

边界与辩证

- 上图是“严肃系统”的下限而非上限：A-MEM 等实验系统没有独立巩固层，靠写入时演化替代。
- 异步写路径的最终一致性问题没有银弹，读屏障只是把不一致的窗口从“无限”缩到“一次 drain”。
- 多 Agent 记忆的共享粒度（谁能写谁的记忆）目前全靠应用层自觉，框架不提供权限模型。
- MemoryOS/MemTree 的研究数字（+48.36% F1 等）来自论文自报口径，横评环境与生产环境的工作负载分布不同，直接照搬预期不可靠。

第二部分 · 机制

第 4 章 写入：抽取、编码与冲突消解

写入是记忆系统里 LLM 调用最密集、设计分歧最大的一段。核心分歧是“冲突什么时候消解”——围绕这个问题，五个源码实测系统给出了五种答案。

4.1 先看坐标系：抽取的三种范式与一个关键实证

2025 年综述把所有写入侧的“抽取”归为三种范式（[arXiv:2504.19413](#)）：**直接归档**（零处理存原文+时间戳，MemoryBank、MemGPT）、**摘要式抽取**（LLM 生成摘要/关键词/标签，A-MEM、Mem0，依赖 prompt 工程）、**图式抽取**（提 SPO 三元组建图，Mem0^g、Zep）。三种范式的代价结构完全不同：直接归档零抽取成本但检索噪声大；摘要式每条 1 次左右 LLM 调用、有语义损失；图式抽取最贵（多次调用）且结构损失最大。

三种范式谁更好，2026 年有了一条可以被复算的实证线：**逐字存储系（MemPal）在 LongMemEval-S 上以纯逐字文本拿到 96.6% R@5，加混合检索与重排后 99.4%；而 Mem0 在同一机构测的 ConvoMem 上只有 30–45%——结论指向“LLM 事实抽取会丢掉逐字存储保留的信息”**。出处：[thesingularity.dev/blog/take-a-look-its-in-a-book](#)（快照 2026.03，含口径注记）。同时横评发现**保留原始消息的 Mem0 优于纯图三元组的 Mem0^g**（[arXiv:2504.19413](#)）。两条独立证据指向同一判断：**抽取是有损压缩，无损层（原文）必须保底**。这条判断直接改写入层的设计清单——无论选哪派，都应同时落一份带时间戳的原始消息表（成本极低），它既是抽取错误时的恢复源，也是第 2.4 节“可溯源”维度的数据基础。

另一个容易被忽略的写入细节是**时间锚定**：对话里说“下周去新加坡”，写入时必须落成绝对日期。Mem0 的做法是在抽取提示词里强制“相对时间按观察日期（observation date）落地为绝对日期”（`ADDITIVE_EXTRACTION_PROMPT`，prompts.py:468 实测）——没做这一步的系统，所有“下周/昨天”类记忆在一个月后全部语义错乱。这是自建写入层时性价比最高的一条规则。

4.2 Mem0：八阶段流水线与"只做加法"的转向

对 mem0 v2.0.20 源码逐行实测（mem0/memory/main.py，共 3868 行），Memory.add()（行 760 起，infer=True 默认路径）是一个八阶段流水线：

1. **Phase 0** 上下文：从 SQLite messages 表取最近 10 条原文（行 918-921）；
2. **Phase 1** 既有记忆召回：把新消息嵌入后向量检索 top 10，UUID 映射成整数 id 防 LLM 幻觉（行 923-938）；
3. **Phase 2 LLM 抽取**：用 ADDITIVE_EXTRACTION_PROMPT（prompts.py:468）输出 JSON 事实列表，含 12 个 few-shot，强制相对时间按观察日期落地为绝对日期；用户提示词由 generate_additive_extraction_prompt()（行 1016）拼装 Summary/Last k Messages/Recently Extracted/Existing Memories/New Messages/Observation Date 七段；
4. **Phase 3 批量嵌入**：embed_batch，失败逐条重试（行 991-1003）；
5. **Phase 4/5 去重**：hashlib.md5(text) 命中库内或本批 hash 即跳过（行 1005-1039）；
6. **Phase 6 批量持久化**：写入向量库 + SQLite history 表记录 ADD 事件（old_memory/new_memory/event 三元组可回放）；
7. **Phase 7 实体链接**：spaCy 抽实体（11 类实体：PERSON/ORG/GPE/LOC/FAC/PRODUCT 等），写入独立实体集合，精确文本匹配优先、语义匹配阈值 0.95，命中则向记忆的 linked_memory_ids 反向索引追加（行 1086-1190）；
8. **Phase 8 保存会话消息**。

值得注意的架构转向：老版 Mem0 用一条"四操作提示词"（ADD/UPDATE/DELETE/NONE，DEFAULT_UPDATE_MEMORY_PROMPT，prompts.py:176）让 LLM 在写入时决定每条事实的增删改；v2 OSS 主链路换成了**只做 ADD 的加法抽取**（v3 提示词），冲突消解被推迟到检索与应用策略侧。这是用写入简单性换取检索复杂性的典型权衡——第三方实测也确认了这一转向（"single-pass, ADD-only extraction: one model call, no UPDATE or DELETE during extraction"），出处：dev.to/jangwook_kim_e31e7291ad98/mem0-vs-zep-vs-letta-the-production-cost-of-agent-memory-52ic。

为什么"只做加法"是理性选择：写入路径上的 DELETE/UPDATE 判断是整条链路里错误率最高的 LLM 决策——判错删除不可逆，而多存的冗余至少还能在检索期过滤。这符合"先记后整理"的普遍工程逻辑（对应第 6 章巩固）：写入追求不丢，整理追求不乱，两个目标分开处理比挤在一次调用里更稳。代价是库内冗余单调累积，Mem0 用 md5 哈希兜底明显重复，其余靠检索期噪声容忍。

实体"图谱"的真相：Mem0 的实体层不是图数据库，而是**第二个向量集合**存实体向量 + linked_memory_ids 反向索引（main.py:638-643）。检索时查询实体命中后回查记忆 id。宣传语境里的"graph memory"与严格意义的图存储有距离——这一点第 14 章会再回到。另有视角改写细节：当 filters 只有 agent_id 时追加 AGENT_CONTEXT_SUFFIX（prompts.py:947），把事实改写成"Agent 得知/Agent 推荐"视角。

4.3 Graphiti：把"事实的时间性"编码进数据模型

Graphiti 0.30.2（getzep/graphiti，README 自述"A Framework for Building Temporal Knowledge Graphs"）的写入核心是 add_episode()（graphiti.py:1043），13 步链路（实测）：

1. 取历史 episode 作抽取上下文（retrieve_episodes，last_n=RELEVANT_SCHEMA_LIMIT）；
2. 构造 EpisodicNode（created_at=now、valid_at=reference_time）；
3. **抽节点**：extract_nodes（node_operations.py:70）；
4. **节点对齐**：resolve_extracted_nodes（627）——语义召回候选 → 确定性相似度去重 → LLM 去重，三级递进；
5. **抽边**：extract_edges（edge_operations.py:117）；
6. **边对齐**：resolve_extracted_edges（325）——按(source,target,fact 归一化) 精确去重 → 生成 fact_embedding → 并行取两端点间已有边作重复候选 → 两次混合检索（端点内 + 全图找矛盾候选）→ 逐边 LLM 判重/判矛盾；
7. **时序失效判定**：_mark_invalidated_edges（约 540-573 行）：时间区间不重叠不构成矛盾；旧事实 valid_at 早于新事实 valid_at 时，旧事实标记 invalid_at=新事实.valid_at、expired_at=now；
8. **时间戳抽取**：_extract_edge_timestamps（576）用轻量 LLM（ModelSize.small）从 fact + reference_time 抽 valid_at/invalid_at；
9. **节点属性/摘要**（extract_attributes_from_nodes，只传 new_edges 避免重复事实）；
10. **落库**（Episodic -[:MENTIONS]-> Entity 等）；
11. **可选社区增量**（update_community）。

```
# 时序失效核心逻辑 (edge_operations.py 实测节选)
if (edge_invalid_at and resolved_edge_valid_at and edge_invalid_at <= resolved_edge_valid_at) \
    or (edge_valid_at and resolved_edge_invalid_at and resolved_edge_invalid_at <= edge_valid_at):
    continue # 时间区间不重叠, 不构成矛盾
elif (edge_valid_at and resolved_edge_valid_at and edge_valid_at < resolved_edge_valid_at):
    edge.invalid_at = resolved_edge.valid_at # 旧事实在新事实生效时失效
    edge.expired_at = edge.expired_at or utc_now()
    invalidated_edges.append(edge)
```

每条 EntityEdge 携带四个时间字段 (edges.py:263 实测): valid_at (事实何时为真)、invalid_at (何时不再为真)、expired_at (系统何时发现)、reference_time。这套设计让"谁现在任加州司法部长"这类问题可以按时间区间精确回答, 而不是把新旧两条事实同时塞给模型。docstring 明确强调: episode 必须顺序添加并 await (1120-1122 行)——写入吞吐天然受限于抽取链路的顺序性。

时序失效的适用边界值得单独说明: 它解决的是"事实被新事实取代" (搬家、换职位), 不解决"事实只在某时段为真且无取代者" (合同 3 月到期, 4 月既没有新合同也没有失效通知)——后者需要应用层主动写失效事件。它也不解决"事实从未改变但当初就抽错了", 那是数据质量问题, 失效链只会忠实地传播错误。给自建者的建议: 即使不用 Graphiti, "创建时间 + 生效时间 + 失效时间"三字段也应该出现在任何事实表的设计里——事后补时序, 成本是数据迁移级的。

论文: Zep: A Temporal Knowledge Graph Architecture, arxiv.org/abs/2501.13956 (Graphiti README 第 88 行引用; 注意早期外部资料引用过 2407.08887, 以仓库 README 为准)。

4.4 LangMem: 热路径与后台反思两条写姿势

LangMem 0.0.30 (langchain-ai 官方记忆层) 源码实测把写入分为两条互相独立的姿势:

- **热路径:** Agent 在对话中主动调用 manage_memory 工具 (knowledge/tools.py), 同步写 BaseStore——什么时候记、记什么, 由模型自己决定; namespace 是带占位符的 tuple 模板, KeyError 时抛出并列出可用 key 的配置错误。
- **后台路径:** MemoryStoreManager (knowledge/extraction.py, 2114 行) 在对话后异步跑, 四步: ① **多查询召回**——若配了 query_model, LLM 生成多个并行子查询, 否则用指数膨胀窗口 (最近 1/2/4/8... 条消息块); ② **存量排序**——按 score 降序取 query_limit 条, 稳定 id = uuid5(NAMESPACE_DNS, namespace+key); ③ **抽取合并**——交给基于 trustcall 的 MemoryManager 做 insert/update/delete 决策 (多轮自反思, i≥1 时追加 Done 工具); ④ **diff 写回**——与存量做内容比较, 只写真正变化的项。

它的防抖设计值得抄走: ReflectionExecutor.submit(payload, after_seconds=N) 对同一 thread_id 的新任务会取消旧任务——用户连续发言时不重复抽取; 实测版用 PriorityQueue + daemon 线程, 远端版映射到 LangGraph Platform 的 multitask_strategy="rollback"。出处: 仓库 github.com/langchain-ai/langmem。

两条写姿势对应的其实是两种信任模型: 热路径把"记忆什么"的判断交给运行中的 Agent (可能漏记、可能被注入操纵——见第 15 章), 后台路径把判断交给确定性流水线 (稳定但不知当下语境)。LangMem 同时提供两条并在文档里建议"热路径记关键约定、后台路径补全画像", 这个分工比二选一更接近生产实际。

4.5 CrewAI: 写入管线 Flow 化与"合并四象限"

CrewAI 1.15.21 源码实测 (重要前提: **新版已废弃 short_term/long_term/entity 三分模型**, 旧三类只在事件注释里残留字符串) 改为单一 Memory + 分层 scope。写入是一条五步 EncodingFlow (encoding_flow.py 实测):

```
Memory.remember(content)
└─ _submit_save(_encode_batch, [content]) # 单 worker 线程池串行化
    └─ EncodingFlow.kickoff(...)
        ├── batch_embed() # 一次性批量 embed
        ├── intra_batch_dedup() # 批内余弦去重 (阈值 0.98, 几乎只挡完全重复)
        ├── parallel_find_similar() # 并发查相似记录 (≤8 线程)
        ├── parallel_analyze() # 并发 LLM: 字段解析 + 合并决策
        └─ execute_plans() # 批量 re-embed + 删/改/插
```

最有趣的是**合并四象限**（analyze.py 实测）：A=字段齐+无相似→快插；B=字段齐+有相似→1 次合并调用；C=字段缺+无相似→1 次字段解析；D=字段缺+有相似→2 次并发调用。LLM 调用次数按输入状态动态伸缩，而不是固定流程——这是对"每次写入固定 1-N 次 LLM 调用"的成本优化思路。它背后的洞察是：**写入成本的大头不是存储而是判断，判断量应当随"新信息与库内的关系"伸缩——全新信息（象限 A）不该付合并的钱，冲突信息（象限 B/D）才该付。**

4.6 Cognee：写入与富化解耦的流水线

Cognee 的 cognify 主流水线（api/v1/cognify.py 实测）五任务：classify_documents → extract_chunks_from_documents → extract_graph_and_summarize（LLM 抽图 + 摘要并行）→ add_data_points（落库）→ 可选 record_provenance / detect_contradictions / resolve_temporal_contradictions（默认全关）。

它的独特设计是把"记忆维护"后置到独立的 memify 流水线，且可对已有图反复后处理（memify_task_registry.py 实测按名调任务）。两个值得记录的细节：

- **确定性 id**：DataPoint 的 identity_fields 决定 id = uuid5(NAMESPACE_OID, "{ClassName}:{' | '.join(norm(v))}")——同名实体跨次抽取天然可去重合并；index_fields 决定向量化字段，向量 collection 按 {TypeName}_{index_field} 命名（如 Entity_name）；
- **双时态预留**：DataPoint 基类含 valid_to 字段（"被取代时刻"）与 truth subspace 字段——图派系统对时间性的处理在 Cognee 里是模型层的预留而非 Graphiti 那样的完整机制。

"写入与富化解耦"在工程上有明确的适用判据：**富化任务会随算法迭代而重跑**（换个 embedding 模型、换个抽取提示词就要重建），把富化和写入耦死意味着每次迭代都全量重灌；解耦后只需重跑 memify。代价是中间态管理复杂度（哪些数据点富化到哪一版）。这和数仓里"ODS/DWD 分层"是同一个思想。

4.7 A-MEM：写入时演化（evolution-on-write）

A-MEM 把冲突消解压进写入瞬间：add_note() → 找 5 个最近邻 → 套演化系统提示词 → LLM 返回 JSON，两种 action：strengthen（给新记忆加链接/改标签）或 update_neighbor（改写邻居的 context/tags）；演化计数达阈值（默认 100）触发 consolidate_memories() 全量重建 Chroma。理念激进：**记忆是可变的（mutable），不是只追加的（append-only）**。但官方精简仓库的实测暴露了落地陷阱：链接存的是枚举序号而非 UUID，search_agentic 用 memories.get(link_id) 永远取不到——**链接扩展在主路径上实际失效**；update_neighbor 按列表下标回写，dict 顺序一变就张冠李戴；evolution_history 字段存在但代码从不 append。教训：**让 LLM"自主改写记忆"的前提，是身份引用系统本身可靠——UUID/稳定 id 是一切 agentic memory 的地基**（Cognee 的确定性 id 与 Mem0 的整数映射是正面对照）。

4.8 OpenHands 与 Letta：文件即记忆与 Agent 自写

两个补充样本拓宽写入光谱的两端。**OpenHands**（coding agent，TS 镜像实测）的"写入"是把状态直接落成仓库文件：跨会话笔记写 .openhands/memory/、会话计划写 PLAN.md——写入动作就是文件创建，没有抽取管线、没有向量编码，可 diff 可 review 可回滚。当 Agent 的工作目录本身就是持久存储时，"写入"退化为"写文件"，这是光谱上最简的一端。**Letta** 则是另一端：写入完全由 Agent 在对话流内以 function call 完成（core memory edit / archival insert），写什么、何时写由模型自主决定——最灵活，也把"记忆质量"完全押在底座模型的元认知上。两者的共同启示：**写入机制没有标准答案，答案取决于"谁最清楚什么值得记"**——工作目录可寻址时是文件，模型元认知强时是自主工具调用，其余中间态才是抽取管线。

4.9 五种写策略的本质差异（汇总）

策略	代表系统	冲突何时消解	LLM 调用模式	代价
写入时消解 (四操作)	旧版 Mem0、 LangMem 后台	写入当下，LLM 判 ADD/UPDATE/DELETE	每次写入 1 次（决策重）	慢且易错删
只加法，检索 时消解	v2 Mem0	推迟到检索与应用策略	每次写入 1 次（仅抽取）	库内冗余累积，靠哈希兜底
时序双写，区 间失效	Graphiti/Zep	写入时做时序判定，旧事实标记失 效不删	每 episode 多次（节点+边+去 重+时间戳）	链路长、顺序写入吞吐受限
状态自适应	CrewAI EncodingFlow	写入时按四象限动态决定	0-2 次/条（按字段与相似度）	阈值调参敏感

演化式改写	A-MEM	写入时改写邻居	每次写入 ≥1 次 + 定期全量重建	身份系统不可靠时改写即事故
直接落盘 / Agent 自写	OpenHands、Letta	不消解（文件可覆盖）或由 Agent 自行改写	0 次（OpenHands）或对话内按需	OpenHands 无冲突检测；Letta 依赖模型元认知

研判

- 在现有证据下，"只加法 + 检索期消解"是写入侧的主流收敛方向：写路径简单可并行，符合异步化趋势。
- Graphiti 的时序失效是当前对"事实会变"问题最完整的工程答案，适合事实高频变更的场景。
- 防抖/批量/幂等（哈希去重、确定性 id、幂等标记如 Cognee 的 `feedback_weights_applied`）是所有写路径的公共工程底座，与流派无关。
- "原文保底 + 抽取加速"应当是默认架构：纯抽取派（含图式）的信息损失已有两路独立实证，原文层是唯一不可再生的数据。
- 相对时间锚定绝对日期，是所有写路径中投入产出比最高的一条规则。

边界与辩证

- 只加法策略的记忆库会单调膨胀，长期运行下去检索噪声是否可控，公开证据不足。
- "逐字存储优于抽取"的实证来自检索召回口径（R@k），端到端 QA 准确率上抽取派的摘要简洁性仍可能占优（第 5.5 节详解指标差异）；两派之争尚未到下定论的时候。
- 时序失效依赖 LLM 抽取时间戳，抽取错误会把"失效链"本身带歪；Graphiti 文档未给出该环节的独立错误率。
- HippoRAG2 的实测教训：每 chunk 两次 LLM 调用（NER + RE）的离线抽取在"每秒几条"的高频对话记忆上根本不适用——写路径成本上限由业务写入频率倒逼决定。
- CrewAI 的 `batch_dedup_threshold=0.98` 意味着批内几乎不去重，跨批次去重依赖合并象限——阈值背后是被低估的调参成本。
- LangMem 热路径的自主写入存在被提示注入操纵的风险（诱导 Agent 写入恶意"约定"），自主性越高的写入姿势越需要配额与审计。

第二部分 · 机制

第 5 章 检索：从向量到图

5.1 纯向量检索的天花板

向量检索回答"语义相近"，回答不了三个问题：**这件事什么时候为真**（时间性）、**这条记忆和那条有什么关系**（多跳关联）、**这个词在文档语境里是否就是那个意思**（词义消歧）。所以 2026 年所有进入主流的记忆系统都不再是纯向量——四大框架"全部在向量之上叠加了关键词、实体或图信号"（Particula Tech 实测综述，[particula.tech](#)）。

纯向量的失效可以用公开数据直接演示：LongMemEval 的"知识更新"类问题（用户偏好变了）要求系统只返回最新状态，而语义相似度上"旧偏好"和"新偏好"几乎等距——没有时间信号或时序失效标记的纯向量系统在这类题上结构性失分。这也是为什么第 4 章的时序字段（`valid_at/invalid_at`）必须在**检索期**被消费：写入期打的标，检索期不用等于白打。

5.2 五个实测样本

样本一：Mem0 的三路混合打分

Mem0 `search()`（`main.py:1379`）实测：语义召回过采样（`internal_limit = max(limit*4, 60)`）+ BM25 稀疏检索（查询先 `lemmatize`, Qdrant `using="bm25"`；原始分按查询词长分档 `sigmoid` 归一化，≤3 词档 (5.0, 0.7) ... >15 词档 (12.0, 0.5)）+ 实体加成（查询实体查实体库 top 500，相似度 ≥0.5 者按 `boost = similarity × 0.5 × 1/(1+0.001(n-1)2)` 加成其链接记忆，n 为加成实体数）。三路分数相加后按信号存在与否自适应归一化（`combined = (semantic + bm25 + entity_boost)/max_possible`，`max_possible` ∈ {1, 1.5, 2, 2.5}），语义分低于 `threshold` 直接淘汰，可选 `rerank`。工程上值得注意的是**过采样**：先用大候选池再压缩，是混合检索的标准姿势。

样本二：HippoRAG2 的 Personalized PageRank

受海马体索引理论启发（少量线索经海马体扩散激活皮层记忆）：query 先按 query_to_fact / query_to_passage 两种 instruction 分别编码 → 事实向量召回 → LLM rerank（DSPyFilter）保留 top-5 事实 → 取主语宾语做图种子（实体权重 = fact_score / 出现篇章数，高频实体按篇章数降权，等价 IDF）→ personalized_pagerank(damping=0.5, reset=种子分布) 在无向图上扩散 → 只取篇章节点的分数排序（HippoRAG.py:693 起，实测）。rerank 后为空则退化为纯稠密篇章检索。另有 retrieve_ircot：检索↔推理交替的多跳迭代。关键默认参数（config_utils.py 实测）：

参数	默认	含义与敏感性
damping	0.5	PPR 阻尼（teleport=0.5）；0.5 vs 0.8 结果差异大
linking_top_k	5	每步保留的图种子数；太小漏多跳中间实体
passage_node_weight	0.05	DPR 分在 reset 向量中的配比，决定稠密/图信号权重
synonymy_edge_sim_threshold	0.8	同义边余弦阈值；低则错连、高则孤岛
retrieval_top_k	200	最终取多少篇 passage

论文：HippoRAG 2: From RAG to Memory, arXiv:2502.14802 (ICML'25), arxiv.org/abs/2502.14802; v1 为 NeurIPS'24 的 arxiv.org/abs/2405.14831。

样本三：Graphiti 的四路并行 + 四种重排器

search.py:98 实测：BM25 全文 / 余弦相似度 / BFS 图遍历 / 社区检索四路并行（semaphore_gather），每路取 2*limit，候选取并集后按配置重排——RRF（倒数排名融合）、MMR（最大边际相关性，去冗余）、cross_encoder（LLM/交叉编码器精排，先 RRF 取前 2*limit 再精排）、node_distance（以某实体为中心的距离重排）。查询带 center_node 时自动切换配方（EDGE_HYBRID_SEARCH_NODE_DISTANCE）；预制配方库 search_config_recipes.py 含 COMBINED_HYBRID_SEARCH_CROSS_ENCODER、EDGE_HYBRID_SEARCH_RRF、NODE_HYBRID_SEARCH_RRF 等。

RRF（倒数排名融合）：不比较不同检索器的原始分值（BM25 分与余弦分不同分布，不可直接相加），只看每路结果的排名，按 $\sum 1/(k + rank)$ 融合。Cognee 的实测实现 (ranking.py)： $r_{rf_score} = \sum 1/(r_{rf_k} + rank + 1)$ ，其中 $r_{rf_k} = clip(20 + 2*top_k, 30, 60)$ ，再乘重要性因子 $0.75 + 0.5*importance_weight$ 。RRF 是多路检索的“通用货币”。

样本四：CrewAI 的置信路由检索

CrewAI RecallFlow (recall_flow.py 实测) 是唯一把“要不要继续搜”交给置信度的：**shallow 路径**——embed → 向量搜索 → 复合打分；**deep 路径**（默认）——短查询（<250 字符）跳过 LLM 分析，长查询由 LLM 分解为 1–3 个子查询 → LLM 从 scope 树选候选 scope（≤20）→（子查询×scope）并行向量搜索 → **置信路由**：置信度 ≥0.8 直接合成结果，0.5–0.8 触发深度探索（预算默认 1 轮，把 top5 喂 LLM 抽取后重搜），<0.5 继续循环直至预算耗尽 → 去重 + 复合打分排序。复合打分公式（types.py 实测）：

```
composite = 0.5 * semantic + 0.3 * recency_decay + 0.2 * importance
recency_decay = 0.5 ** (age_days / 30)      # 半衰期 30 天
# 默认权重 semantic 0.5 / recency 0.3 / importance 0.2, importance 默认 0.5
```

这是把 Generative Agents 三因子思想（第 10 章）工程化为可配置公式的直接案例。

样本五：MemR3 的闭环检索（研究前沿）

检索侧最新的研究动向是把“检索一次”改成“检索到够为止”：MemR3（2025, [arXiv:2602.14038](https://arxiv.org/abs/2602.14038) 附录收录）把标准的“检索→作答”管线改造成闭环序贯决策——路由器在 retrieve / reflect / answer 三个动作间动态选择，核心是一个**全局证据缺口追踪器**：显式维护“已验证证据”与“缺失信息”两个清单，每轮检索后更新，缺口闭合即提前停止。作为即插即用控制器，它兼容任意现有记忆后端。这与 CrewAI 的置信路由是同一思想（检索深度自适应）的两个实现层次——CrewAI 靠一次置信度打分，MemR3 靠显式证据状态机。工程意义：对多跳问题，“检索几次”本身应是被管理的变量，而不是固定 top-k。

5.3 重排器的分层：便宜信号在前，贵的在后

五个样本共享一个结构：**便宜信号做召回（BM25/向量），昂贵信号做精排（cross-encoder/LLM）**。这一分层是成本工程的核心——cross-encoder 精排只对 top 2×limit 做，LLM rerank 只对 top-5 做。Generative Agents（斯坦福 Smallville）的三因子检索是这个结构的 2023 年先驱：recency × 0.5 + importance × 3 + relevance × 2 加权归一化取前 30（retrieve.py 实测，importance 权重最高），

其中"importance"就是一次 LLM 打分 (poignancy 1-10)。2026 年的实证补了关键一刀：**分层的管线确实是性能来源**——同一机构的多层管线 (向量+词法+分 session 嵌入+cross-encoder 重排) 在 LoCoMo 上拿到 96.5% R@10，比公开的 hybrid-only 基线 (88.9% R@10) 高约 8 个点 (单点观察，作者自述)。出处：[thesingularity.dev](#)。

5.4 OpenHands 的提醒：检索有时可以没有

OpenHands (coding agent, TS 类型镜像实测) 的记忆策略没有向量检索：**condenser 是对"喂给 LLM 的视图"做有损裁剪，不是删数据库**——全量事件留存在 conversation store，只是从窗口里"换出" (CondensationEvent.forgotten_event_ids)。默认开启 (enable_default_condenser=true)，且 condenser 是一次**独立计费的 LLM 调用** (metrics 按 "condenser" 桶单列)。对一个工作目录就是记忆的 coding agent 来说，**文件系统本身是可寻址的存储，检索退化为 ls/grep**——这是"检索问题消失"的边界案例，值得纳入架构考量。更一般的判据：当记忆总量小到可以全量装进窗口、或空间结构可枚举 (文件树、数据库 schema) 时，语义检索层可以整体省掉；反之则必须建。

5.5 读检索数字之前：三个必须核对的事

检索是记忆领域数字水分最大的环节，读任何报告前先核对三件事：

核对项	问题	实例
◎ 指标口径	R@k (检索召回) ≠ QA 准确率 (端到端答对)。一个系统把正确段落排到第 1，LLM 读完仍可能答错；反之亦然。两个数字经常相差 5-15 个点	Mastra 宣称 LongMemEval 94.87%，实为 GPT-5-mini 作答的端到端 QA 准确率；MemPal 的 96.6% 是 R@5——两个数字不可比
◎ k 值与数据形状	k 必须小于数据最小单元数，否则检索被"包圆"，分数只反映重排器的阅读理解	LoCoMo 每段对话仅 19-32 个 session，top-k=50 时答案必在候选池里；诚实口径是 top-10 (96.5% R@10 的报告者专门注明此点)
◎ 判官模型与判分规则	端到端分数由 LLM judge 决定：换 judge、换 prompt、去掉对抗题，同系统分数可差 10-25 点	Mem0-Zep 之争：同一系统被测出 65.99% / 75.14% / 58.44% (去对抗题重跑) 三个数

口径与实例来源：[maximem.ai/blog/best-ai-agent-memory-platforms-2026](#) (该文同时开源了 MIT 协议的评测 harness，可自跑四家系统)、[thesingularity.dev](#)、[geenalpai.com](#)。三件事也分别对应第 14 章基准通胀问题的技术根源。

补一个分题目的观察 (同一来源，供参考口径)：在 LongMemEval-S 的分类别 R@10 上，开源可复现报告显示 single-hop 97.5%、multi-hop 97.2%、temporal 95.3%，但 open-domain 只有 84.8%——**需要外部知识补全的开放域问题，是记忆检索的结构性短板** (记忆库里没有的答案，检索层无能为力)，这类需求应导流到 RAG/工具调用而非记忆层。这提醒我们给记忆系统划定能力边界：记忆管"你们之间发生过什么"，不管"世界上是什么"。

研判

- 混合检索（稠密+稀疏+图/实体）已是 2026 年的事实标准，纯向量只适合原型。
- PPR/图扩散对多跳问题有结构性优势，但对单跳事实查询收益有限且参数敏感（HippoRAG 实测）。
- CrewAI 的置信路由与 MemR3 的证据缺口追踪代表"检索深度自适应"的正确方向，但探索预算与阈值需按业务重调。
- 读任何检索数字前先核对指标口径、k 值、判官——三件事不过关的数字不进入选型依据。
- 开放域问题不属于记忆层的能力范围，架构上应显式分流。

边界与辩证

- 图信号的维护成本常被低估：三元组抽错会沿图传播，PPR 把错误"扩散"到错误篇章，rerank 只看 top-5 时错种子=错全链（HippoRAG 实测坑点）。
- 多路融合的分值不可比，各家的归一化/融合公式均为经验调参，跨系统不可复用。
- "混合检索一定更好"缺乏受控对照：Braintrust 2026 分析提醒，记忆检索不当（过时/不相关/延迟）反而降低任务表现（jakeinsight.com）。
- Mem0 的三路打分要求向量后端支持关键词检索，27 种 provider 中仅 16 种覆写 keyword_search()，未覆写时静默退化为纯语义——部署后必须验证。
- 分层的 +8 点增益是单系统单数据集的单点观察，且报告者自身是相关方（ghola 作者），引用时保留折扣。

第二部分 · 机制

第 6 章 遗忘、压缩与巩固

Anthropic 在发布 memory tool 时的表述值得刻在墙上："记忆系统同时也是一个遗忘系统"。不会遗忘的记忆会腐烂——过时事实、失效偏好、无细节堆积后，检索质量与 token 成本同时恶化。遗忘在工程上有五种形态（比上一版多一种）：

形态	机制	实测实例
物理删除	硬删除向量/行	Mem0 delete 写 DELETE 历史 (is_deleted=1) 后删向量，并从实体库摘链（main.py 实测）
时序失效	标记 invalid_at，不再删除	Graphiti 旧事实失效进 invalidated_edges，保留完整历史链
压缩/折叠	旧消息折叠为摘要	LangMem summarize_messages；Anthropic compaction；OpenHands condenser
视图裁剪	不删数据，只从窗口视图移除	OpenHands CondensationEvent.forgotten_event_ids：全量事件留存 store，"遗忘"仅指离开 LLM 视图
衰减	按访问时间/频率降权	Generative Agents 的 recency 指数衰减（decay=0.995）；CrewAI 半衰期公式 $0.5^{(age_days/30)}$ ；A-MEM 的 retrieval_count/last_accessed 字段

第五种形态（衰减）之外，研究侧还有一个更彻底的变体：**MemoryOS 的热度驱逐**——MTM 条目按"访问次数 + 对话页数 + 时间衰减"算热度分，低于阈值直接淘汰出中期记忆、高者升入长期记忆。它与 CrewAI 半衰期的区别是把"使用频率"显式加入，遗忘决策从"多旧"变成"多旧 × 多没用"。对自建者，这两个信号（age、access_count）缺一不可：只按时间衰减会忘掉"旧但常用"的核心约定，只按频率会永久保留"早期高频后来无关"的死记忆。

6.1 压缩的两个实测样本

LangMem：运行式摘要（running summary）

summarize_messages（short_term/summarization.py，861 行实测）：每轮把消息列表喂给摘要函数，超过 token 阈值就把旧消息折叠。数据结构 RunningSummary(summary, summarized_message_ids, last_summarized_message_id) ——记录"摘到哪了"，增量续摘而非全量重摘。切点选择有讲究：逐条 token 累加定位切点后，若切点最后一条是带 tool_calls 的 AIMessage，把对应 ToolMessage 也

并入摘要（工具调用与结果不可拆）。三条默认提示词（L23-49 原样实测）：初始摘要 "Create a summary of the conversation above"、续摘 "This is summary of the conversation so far: {...}\n\nExtend this summary..."、终态 [System("Summary of the conversation so far: {summary}")]] + 未摘要新消息。短期记忆没有自己的存储后端，只是"消息表的改写器"，改写结果交回 LangGraph 状态图由 Checkpointer 持久化。

一个容易被照抄错的细节：增量续摘比全量重摘便宜得多，但会**累积摘要误差**（摘要的摘要）——LangMem 的缓解办法是保留 `summarized_message_ids`，必要时可回到原文重摘。自建时同理：**摘要必须可溯源到原文 id 范围**，否则错误会在摘要链上无声传播且无法修正。

OpenHands: condenser 的事件视图模型

OpenHands (TS 镜像实测) 把压缩建模为事件流上的一类事件：`CondensationEvent{forgotten_event_ids, summary, summary_offset}` ——被遗忘事件显式列出、摘要带插入偏移。三个工程要点：① condenser 默认开启且 `max_size` 默认不限；② 它是一次独立 LLM 调用，token 与成本单列（metrics 桶 "condenser"）——压缩不是免费的；③ 遗忘是视图级的，conversation store 全量留存，可回溯。Anthropic 的 context editing 是同一思想的服务端实现：上下文将满时自动清除过期工具调用与结果（100 轮搜索任务 token 节省 84%，o-mega.ai 引官方数据）。

6.2 巩固：从"记住"到"记得更好"

巩固 (consolidation) 指对已存记忆的后置加工：合并、抽象、纠错。四个实测样本：

- **反思 (reflection)**：Generative Agents 的做法——累计重要性"预算"（`importance_trigger_curr`，默认 150，bootstrap 实测 250）每写入一条事件就减去该事件的 poignancy，**预算花光即触发反思**：LLM 从最近事件提出 3 个焦点问题 → 三因子检索各召回 30 条 → LLM 生成 5 条高层洞见（格式 "insight (because of 1,5,3)"）→ 以 thought 写回，`depth = 1 + max(证据 depth)`，filling 记录证据节点 id，形成可下钻的抽象层级。这个"重要性预算"设计把"何时整理"变成了确定性规则而非 LLM 自主判断——触发条件可预测、成本可控，比"定期跑 cron"更贴合信息密度。
- **做梦 (dreaming)**：MemOS 2.0.33 的 `dream/` 插件实测：写记忆累积信号，达阈值（默认 100 条 pending）触发六阶段流水线——聚上下文 → 形成"动机"簇 → 召回相关记忆 → LLM 推理出每簇至多一条 insight → 生成"梦日记" → 写回图库。OpenAI 2026 年 6 月给 ChatGPT 上线的 Dreaming 是同一思想的消费级产品：后台跨会话合成记忆、随时间修订时效性事实（"你七月要去新加坡"→"你 2026 年 7 月去了新加坡"），官方报告事实召回从 41.5% 升至 82.8%。出处：taskade.com/blog/agent-memory-history、o-mega.ai。两者共同的设计约束值得注意：**insight 每簇至多一条、dream 日记可审读**——后台自动改写记忆的信任问题，靠"产量克制 + 产出可见"缓解。
- **实体对齐/合并**：Cognee 的 `consolidate_entities` 实测——名字向量相似度 0.85 阈值 + 归一化名相等做并查集聚簇（分块扫描避免 N×N，每节点只看 top 10 近邻），**度数最高者为 canonical**（其次 `created_at` 最老、名字典序），重指向边后删除重复节点并清向量；支持 `dry_run` 只算计划不改库。"度数最高者胜出"是个聪明启发：被引用最多的实体最可能是正名；而 `dry_run` 接口说明作者清楚这个操作的破坏性——**任何不可逆的巩固操作都应先能预览计划**。
- **反馈巩固**：Cognee 的 `apply_feedback_weights` 实测——用户 1-5 分归一化后以 EMA 流式更新节点/边权重：`weight' = weight + 0.1 × (norm - weight)`，clip 到 [0,1]，幂等键防重复应用；配套 `apply_frequency_weights` 按 QA 实际用到的图元素 +1.0。这是把"用得多的记忆更可信"显式写进排序。

6.3 该忘什么、不该忘什么：遗忘的分界线

工程实践（ChatForest 综述，chatforest.com/guides/ai-agent-memory-patterns）给出的分界线：用户纠正（"别再这样做"）、安全相关信息（权限/合规/边界）、核心身份信息应当**抗衰减**甚至永不衰减；临时事实、过时工具结果、可再生的过程数据应尽快淘汰。这个分界线目前没有任何主流框架内置为策略——它几乎总是应用层的责任。可核查的对照：CrewAI 的 MemoryRecord 有 `private` 字段（隐私过滤）但无"抗衰减"标记；MemOS 的记忆状态机有 `activated/archived/deleted` 但无 `protected` 类别。

给自建者的最小可行方案：在记忆记录上加一个 `durability` 枚举字段（`ephemeral / standard / protected`），衰减与驱逐逻辑按枚举分流，`protected` 记录只允许显式用户操作删除。**这条字段的窗口期很短**——一旦系统运行数月、历史数据没有衰减标记，再想区分"该忘的"和"不该忘的"就没有数据基础了。这是和第 2.2 节 `access_count` 同级的"事后补不了"字段。

6.4 遗忘的合规面：被遗忘权进入工程清单

2026 年 8 月 EU AI Act 透明度要求全面生效后，记忆治理从"最佳实践"变成"合规项"：用户有权知道 Agent 维护了关于自己的持久信息、有权复核与更正、有权要求删除。出处：clawdbot2.in/how-does-ai-agent-memory-work-2026。这对架构的直接要求：① 记忆必

须按用户可枚举（scope 隔离在这里第一次成为合规需求而非工程洁癖）；ⓐ 删除要能覆盖所有物化形态——向量、图节点、摘要、派生 insight 都要跟着删（Cognee 的 provenance 链在这时变成刚需：没有溯源链，无法判定哪些派生物须一并清除）；ⓑ 留操作审计日志（业界已有 append-only audit log 的实现先例，同上来源）。被动遗忘（衰减）与主动遗忘（删除请求）必须分开实现：前者是质量优化，后者是法律义务，混在一个 TTL 机制里会让合规审计无从下手。

研判

- 遗忘机制的缺失（而非检索算法的优劣）是长周期记忆系统质量分化的首要原因。
- 巩固正在从研究概念变成产品功能（MemOS Dream 插件、ChatGPT Dreaming、Cognee 反馈加权），"后台定期整理记忆"会像"数据库 VACUUM"一样成为标配运维项。
- 视图级遗忘（OpenHands/Anthropic）比物理删除安全：数据可回溯，误压缩可恢复——生产系统优先选它。
- age、access_count、durability 三个字段是遗忘体系的最低数据基础，且都属于"事后补不了"的设计，必须进 schema v1。
- 被动遗忘与合规删除是两条独立管线，EU AI Act 生效后后者在有欧盟用户的场景不可省略。

边界与辩证

- 巩固依赖 LLM 判断"什么重要"，而重要性判断错误不可逆——错误 insight 写回后成为下一轮检索的输入，误差可能滚雪球。
- MemOS Dream 实测局限：信号只存内存、重启丢失，触发策略单一，beta 状态。
- Generative Agents 参考实现存在方向反了的 recency 因子（nodes 按 last_accessed 升序、decay 指数递减，归一化后最久未访问者得分最高，源码实测复算确认）——被引用最多的经典代码也有 bug，复现时应读代码而非只读论文。
- EMA 反馈加权（Cognee）对恶意/错误评分无防御，feedback_influence 默认 0 可能正是出于这个顾虑。
- "被遗忘权"的合规边界仍在演化：本文引用的是评述性来源对 AI Act 的解读，具体义务以法规文本与所在地法律意见为准。

第三部分 · 流派

第 7 章 流派全景：六条技术路线

把 17 个开源仓库的源码与 2026 年产品动态放在一起，Agent Memory 的实现收敛为六条路线。每条路线的核心分歧只有一个问题："记忆的真相"放在什么表示里。

路线	核心表示	代表系统	强项	结构性弱点
ⓐ 向量事实层派	抽取事实 + 向量 (±实体反向索引)	Mem0、MemOS(general_text)、CrewAI Memory	接入简单、生态广、 可插拔后端	时间性与多跳弱，"graph"多为实体反向索引
ⓑ 时序知识图谱派	实体-关系图 + 双时间戳	Zep (Graphiti 内核)	事实失效建模、多跳、可审计	抽取链路长、图库运维重、写入吞吐受限
ⓒ 自编辑记忆派	上下文内可编辑记忆块 + 外部归档	Letta (MemGPT)	Agent 自主管理、生命周期内聚	依赖模型工具使用能力，调试复杂
ⓓ 文件/MCP 派	Markdown 文件即记忆	Basic Memory、CLAUDE.md/AGENTS.md 生态、OpenHands .openhands/memory	可审计 (git diff)、人机共写、零锁定	规模化检索弱、无自动冲突消解
ⓔ 神经-图混合派	三元组图 + PPR 扩散	HippoRAG2、Cognee	多跳检索、离线建库可并行	OpenIE 噪声传播、参数敏感、写入贵
ⓕ 认知仿真派	记忆流 + 三因子 + 反思	Generative Agents (Smallville)	认知闭环完整，学术基线	工程化不足，成本高，2023 年冻结

流派划分综合源码逐行实测与第三方对比（[Particula Tech 2026](#)；[DEV Community 2026](#)）。

7.1 研究侧的第七条路线：记忆资源调度派（2025 年成形）

上表六条路线按"表示"划分，但 2025 年起研究侧出现了一条按"调度策略"划界的路线——不问记忆存成什么，而问记忆如何像系统资源一样被调度。代表系统三个：

- **MemOS** (MemTensor, [arXiv:2507.03724](#))：把明文/激活/参数三种形态的记忆统一封装为 MemCube（内容+来源+版本+使用历史），由 MemScheduler 异步调度增删改，支持三种形态间迁移。2025 年 12 月的 v2.0 "Stardust" 补了多模态记忆、工具记忆与 Redis Streams 调度（[o-mega.ai](#)、[youmind.com](#)）；
- **MemoryOS** (北邮, EMNLP 2025 Oral)：STM/MTM/LPM 三层存储，迁移规则完全显式——FIFO 下移、热度评分（访问次数+对话页数+时间衰减）上移；LoCoMo + GPT-4o-mini 上自报 +48.36% F1 ([arXiv:2505.22101](#))；
- **MemTree** ([arXiv:2505.22101](#) 同批横评)：动态树结构，新信息按语义相似度决定节点合并/新建/分支，顶层粗筛叶层细查——横评中准确率居前、token 开销也居前。

效率轴上还有更新的 **SimpleMem** (2026.01)：语义结构化压缩 + 在线语义合成 + 意图感知检索规划，GPT-4.1-mini 上 F1 43.24（对比 Mem0 34.20、全量长上下文 18.70），同时把推理期 token 消耗最多降 30 倍（数字为论文自报，转引自 [o-mega.ai](#)）。调度派的存在本身是个信号：当"表示"之争进入平台期，竞争轴转向调度与效率——这与 2026 年 1 月 ICLR MemAgents workshop、AAAI 记忆专场的设置一致（同上来源）。给实践者的提示：调度派论文数字多为自报口径且评测环境特殊，参考其**机制设计**（热度升格、三态统一、意图感知路由）比参考其分数更有价值。

7.2 十二系统能力矩阵（源码实测归纳）

六条路线之上，把源码实测过的 12 个系统按第 2 章的五维（正确性/新鲜度/覆盖面/可溯源/可审计）加关键机制摊开——这张表是全书的事实底座，后续章节逐个展开：

系统	记忆表示	检索机制	失效/遗忘	巩固机制	溯源/审计
Mem0	事实向量 + 实体反向索引	稠密 + BM25 + 实体三路融合	物理删除 + history 回放	MD5 去重；无主动巩固	SQLite history (old/new/event)
Graphiti/Zep	时序属性图（双时态边）	BM25+向量 + BFS 四路并行+四种重排	invalid_at 区间失效	节点/社区摘要、label propagation 社区	边级四时间戳 + episode 关联
Letta	核心块+recall+archival 三层	FTS+向量 + RRF 混合 (recall)	块编辑替换 (rethink) + 归档	模型自编辑 (rethink/patch)	block_history (checkpoint/undo/redo)
LangMem	namespace 结构化 Item	多子查询并行向量检索	运行式摘要折叠旧消息	后台 MemoryStoreManager diff 合并	Item created/updated_at；无证据链
CrewAI	scope 路径化向量记录	置信路由 + 子查询分解 + 复合打分	半衰期降权（不删除）	合并四象限；consolidation_threshold=0.85	source 字段；private 标记
MemOS	MemCube 四槽（文本/激活/参数/偏好）	向量/图多后端 + reranker	状态机 activated→archived/deleted	Dream 六阶段 + 反馈合并 (difflib)	sources 链 + version/history
HippoRAG2	三元组图（实体/篇章节点）	事实向量 + LLM rerank+PPR	出处计数归零才删 (delete())	同义边 KNN；无反思	fact_source_counts 逐边出处
Cognee	DataPoint 图（实体/三元组）	向量投影子图+三元组重要性打分	valid_to 预留；无主动遗忘	memify：实体合并/跨连边/频次+反馈加权/层级摘要树	确定性 id (uuid5) + ontology_uri
Basic Memory	Markdown 文件 (Observation/Relation)	FTS+可选向量+重排	文件即状态，无自动遗忘	similar-note 探针提示（不代合并）	git diff + permalink + 前向引用回填

A-MEM	互链卡片节点	向量 top-k + links 展开	无	写入时演化 + 定期全量重建	evolution_history 字段 (未实现 append)
Generative Agents	JSON 记忆流 (ConceptNode)	三因子加权 (O(N) 手写余弦)	expiration 字段 (thought +30 天)	反思：预算触发→焦点→洞见写回	filling 证据链 + depth 层级
OpenHands	事件流 + 文件 (.openhands/memory)	无向量检索 (文件可寻址)	condenser 视图裁剪	无 (依赖文件整理)	事件全量留存 + condensation 事件显式列出

全部条目来自开源仓库逐行实测（版本：Mem0 2.0.20、Graphiti 0.30.2、Letta 0.16.8、LangMem 0.0.30、CrewAI 1.15.21、MemOS 2.0.33、HippoRAG2 main、Cognee 1.5.4 示例、Basic Memory、A-MEM、Generative Agents 2023-08、OpenHands 1.18.0 前端仓）。研究侧补充阵容（未实测、口径为论文/厂商自报）：MemoryOS（三层存储 +48.36% F1 自报）、MemTree（动态树）、SimpleMem（F1 43.24 / 30× token 降低自报）、MemoryBank（艾宾浩斯遗忘曲线衰减）、MemoChat（备忘录迭代循环）、Mastra Observational Memory（Observer/Reflector 观察日志）、MemR3（闭环检索控制器）。

7.3 同构性：六条路线共享的底层机械

2026 年中，四家托管记忆层（Weaviate Engram、Cloudflare Agent Memory、Microsoft Foundry Memory、Mem0）在七周内发布、收敛到同一架构——DEV Community 的评述一针见血：“四家公司七周内发布同一设计，这不是创新，是被托管包装的商品化模式”。真正差异在哲学层：Zep 把真相放进时间区间，Mem0 放进抽取事实，Letta 放进 Agent 自己编辑的块，Basic Memory 放进人可读的文件。抽取-检索机械趋同，表示哲学分化——这决定了选型时比较“检索质量”意义有限，比较“表示与治理模型”才有意义。2025 年 12 月综述的三范式（平面检索型/显式结构型/策略管理型，[arXiv:2603.04740](#) 转述）与六路线的映射：①④属平面型，②⑤与调度派的树属结构型，③⑥属策略管理型——两套坐标系互为投影，前者按表示、后者按组织策略。

矩阵里还有两个反直觉的观察：① 框架的记忆在降级——CrewAI 新版删掉了实体图谱（旧 EntityMemory 退化为 metadata.entities 字符串列表，无图遍历），说明框架层在做减法、把重记忆让给专门系统；② 最简单的反而最全——Generative Agents 这个 2023 年冻结的研究原型，在“溯源+巩固”两维上仍是十二系统里最完整的（filling 证据链 + 预算触发反思），2026 年的多数产品在这两维上仍不及它。第三个观察来自产品侧：托管层与开源内核正在分叉——Zep 社区版弃维、Letta 活跃开发转 TS 仓、Mem0 OSS 与平台版功能边界扩大（时间戳参数仅平台版支持），“用开源版=用阉割版”正在成为默认假设，选型前应逐项核对。

研判

- 六路线是光谱不是壁垒：LangMem 同时是①（BaseStore 事实）与③（Agent 工具自写）；MemOS 同时是①与②（tree 后端带图）。
- 选型的第一问不是“哪家检索准”，而是“你要不要时间性、要不要人可审计、谁拥有记忆的生命周期”——三个问题分别指向②④③。
- 调度派机制（热度升格、三态统一、意图感知路由）比其分数更值得借鉴；“效率轴”（每点 F1 花多少 token）将成为下一阶段的主竞争面。

边界与辩证

- 六分法是分析工具，产品迭代会持续打破它：Letta 活跃开发已转向 TS 仓（letta-code），Zep 社区版已弃维、商业化为 Zep Cloud。
- “商品化”判断基于托管服务层的趋同，开源内核（Graphiti/letta/cognee）仍保持架构多样性。
- 能力矩阵是快照：各系统版本迭代快，引用时注意核对版本号。
- 研究侧补充阵容的数字全部为论文/厂商自报口径，未经独立复现，只作机制参考不作性能依据。

第三部分 · 流派

第 8 章 精讲 · Mem0：向量事实层派

8.1 它到底是什么

Mem0（github.com/mem0ai/mem0，源码实测 v2.0.20，Apache-2.0）是一层与 Agent 解耦的记忆 API：add(messages, user_id=...) 写入、search(query, filters=...) 检索，内部完成抽取、嵌入、去重、实体链接。默认栈为 OpenAI gpt-5-mini 抽取

+ text-embedding-3-small 嵌入 + 本地 Qdrant (/tmp/qdrant , collection=mem0) + SQLite 历史 (~/.mem0/history.db) (main.py / configs 实测)。最小用法 (README):

```
from mem0 import Memory

memory = Memory()

memory.add([{"role": "user", "content": "I prefer invoices as PDF files."}],
           user_id="tenant-a:user-42")

result = memory.search("How should invoices be delivered?",
                       filters={"user_id": "tenant-a:user-42"}, top_k=3)
```

规模参考 (厂商披露口径, 未经独立审计): GitHub 41K+ stars、PyPI 下载 1300 万+, API 调用量从 2025Q1 的 3500 万次增至 Q3 的 1.86 亿次 (约 30% 月增长); 2025 年 10 月完成 2400 万美元 A 轮 (Basis Set Ventures 领投, Peak XV、GitHub Fund、Y Combinator 跟投), 并成为 AWS Agent SDK 独家记忆供应商。出处: o-mega.ai/articles/self-improving-ai-agents-the-2026-guide。读这些数字的**正确姿势**: 调用量证明的是"接入摩擦低" (三行代码集成), 不证明"记忆质量高"——第 14 章的复现争议说明两者必须分开评估。

8.2 多租户命名空间 (工程上最值得抄的部分)

写入强制 user_id/agent_id/run_id 至少其一 (行 817-851 实测), 会话键按字典序拼接为 user_id=..&agent_id=..&run_id=.. (值做 %=& 转义)。还有三个防越权细节: _strip_identity_keys() (行 143) 从用户自定义 metadata 里剥离身份键, 防止 metadata 越权改命名空间; _reject_top_level_entity_params (行 165) 拒绝顶层散传 user_id——命名空间只能走 filters=; _validate_and_trim_entity_id() (行 175) 拒绝内部空格、整数转字符串。多租户隔离不是"加个 where 条件", 而是一整套输入防御。

这套防御的深层设计原则值得提炼: **身份信息是系统级元数据, 永远不该允许用户内容携带**。三个函数分别堵住三条渗透路径 (metadata 夹带、顶层散传、格式混淆)。自建多租户记忆时照此三查即可覆盖绝大多数越权事故, 成本是三个校验函数。

8.3 检索、过滤与静默降级

检索为三路混合 (第 5 章样本一), 过滤条件支持 AND/OR/NOT 与 {eq/ne/gt/...} 比较算子 (行 1467-1475 实测翻译为 Qdrant 的 \$or/\$not)。五个 factory (VectorStore/Llm/Embedder/Reranker/GraphStore) 全部可插拔: 向量库 27 种 provider 中 16 种覆写了 keyword_search() (qdrant/elasticsearch/pgvector/pinecone/milvus/opensearch/redis/weaviate/mongodb 等), 未覆写时启动告警并**静默退化为纯语义检索** (行 543) ——部署后务必确认混合检索是否真的在跑。Reranker 提供五种实现 (cohere/huggingface/llm/sentence_transformer/zero_entropy)。

"静默降级"是这节最值得写进运维清单的词。混合检索与纯语义检索的质量差距可达 5-10 个点 (第 5.3 节), 而降级只表现为一条启动告警——**上线后第一周就该做的验证**: 用一个关键词明确、语义模糊的查询 (如精确产品型号) 测检索, 命中即混合在跑, 不命中即已降级。

8.4 双实现、遥测与 OSS/平台边界

Memory 与 AsyncMemory 是几乎平行复制的两套实现 (行 487 vs 2172), 同步版适合脚本、异步版适合服务。遥测经 capture_event() 埋点 (mem0.init/add/search), 可用 MEM0_TELEMETRY 关闭——合规评估时注意默认开启。历史回放: history(memory_id) 可查看每条记忆的 ADD/UPDATE/DELETE 轨迹。

OSS 与平台的功能边界在 2026 年明显扩大, 选型前逐项核对: 时间戳参数 (timestamp / reference_date) 在 OSS 版被显式拒绝 (行 817-851 实测"OSS 不支持"), 时间性能力基本只在平台版; 图记忆 gating 在平台 Pro 档 (\$249/月, 2026 年 9 月定价, jakeinsight.com); 多级记忆管理 (user/session/agent) 为平台特性 (clawdbot2.in)。Apache-2.0 的 OSS 内核与商业平台"共享 API 形状但能力分档"——genalphai.com 的提醒很实际: 功能与定价边界应以 mem0.ai 为准核实, 不要从 GitHub 推断。

研判

- 适用: 需要快速接入、跨模型可移植、多租户 SaaS 场景。Apache-2.0 可自托管是四大框架中最宽松的。
- 工程成熟度高: 同步/异步双实现、四五个可插拔 factory、history 可回放、命名空间防御完整——其防御设计 (8.2) 与静默降级验证 (8.3) 即使不用 Mem0 也值得照抄。

- 商业势能目前领先（AWS 独家、调用量级），但"接入量"与"记忆质量"是两个变量，后者以第 14 章独立复现为准。

边界与辩证

- 实体层不是真图（第 4 章已证）；图记忆功能在商业平台版 gating 在 Pro 档。
- 基准数字见第 14 章：厂商宣称与独立复现差距大，不应作为选型依据。
- Sync/Async 两套近乎复制的实现（行 487 vs 2172）维护一致性存疑——版本升级时注意行为差异。
- OSS/平台功能分档持续变化，本节边界描述为 2026.09 快照。

第三部分 · 流派

第 9 章 精讲 · Zep/Graphiti：时序知识图谱派

9.1 Graphiti 是内核，Zep 是商业云

Graphiti (github.com/getzep/graphiti, 实测 0.30.2, Apache-2.0, 依赖 pydantic/neo4j/openai) 是开源的时序知识图谱构建库；Zep 是构建在其上的商业服务（专有 "Context Graph Engine"）。两者的边界经常被混淆——**Zep 社区版已弃维，当前 Zep 仓库是面向 Zep Cloud 的示例与集成仓** ([jangwook kim 实测记录](#))。想自托管时序图，用的是 Graphiti + 自备图库（Neo4j 默认，支持 FalkorDB/Neptune；Kuzu 已标记 deprecated）。

9.2 数据模型：五类节点、五类边、双时间轴

实测 `nodes.py` / `edges.py`：节点分 Episodic（原始事件，携带 content 与 valid_at）/ Entity（实体，带 summary——"周边边的区域性摘要"——与 name_embedding）/ Community（社区摘要）/ Saga（情节序列，first/last_episode_uuid + last_summarized_at）等；关系为 Episodic -[:MENTIONS]-> Entity -[:RELATES_TO]-> Entity 加社区边（HAS_MEMBER）与 Saga 边（HAS_EPISODE/NEXT_EPISODE）。核心是 EntityEdge（263 行起）的四时间字段——valid_at / invalid_at / expired_at / reference_time，配合第 4 章的失效判定逻辑，构成"双时态"（bitemporal）模型：**事实的有效时间与系统的认知时间分离**。注意 Saga 节点是较新版本加入的——它把"一段情节（episode 序列）"提升为一等公民并带独立摘要进度（last_summarized_at），这实际是把"会话级摘要"做进了图模型，长对话场景值得单独评估。

9.3 索引与检索的落地细节

Neo4j 方言索引 (`graph_queries.py` 实测)：范围索引覆盖 uuid 唯一性、group_id 分区（多租户）、三个时间索引（valid_at/expired_at/invalid_at 各一条——时序检索是被索引能力支撑的，不是全表扫描）；全文索引覆盖 episode 内容、实体名+摘要、边 fact。向量相似度不在 Neo4j 索引层做，而在驱动层拉候选后用 `vector.similarity.cosine`（FalkorDB 用 `vec.cosineDistance`）。社区检测用 **label propagation**（非 Leiden/Louvain, `graph_utils.py:27` 实测），每簇由 LLM 生成摘要，支持增量并入（新节点并入邻居众数社区）。**选 label propagation 而非更精确的社区算法，是延迟与质量的取舍**——label propagation 近线性、天然支持增量，代价是社区边界质量略逊；对"按用户分区、社区只做摘要辅助"的记忆场景是合理选择，做全局知识组织时未必。

9.4 工程画像与可运行示例

- 检索**：四路并行 + RRF/MMR/cross_encoder/node_distance 重排（第 5 章样本三）；
- 多租户**：group_id 全链路过滤，请求级 driver 解析避免并发串写（issue #1676 的修复）；
- 全异步**：async def 全链路 + semaphore_gather 并发控制；OpenTelemetry 原生追踪（`tracer.start_span('add_episode')`）；
- 约束**：episode 必须顺序添加并 await (`graphiti.py:1120-1122` 实测)——写入吞吐天然受限于抽取链路的顺序性；
- 官方 quickstart** (`examples/quickstart/quickstart_neo4j.py`, 239 行)：插入 Kamala Harris 任加州司法部长（2011–2017）与 Gavin Newsom 任州长两个 episode，然后 `search('Who was the California Attorney General?')` 返回带 valid_at/invalid_at 的 fact——"时间旅行"查询的最小演示。

9.5 数字怎么看：自报基准与三方争议

Zep 的公开数字需要分层读。**第一层，论文自报**：2025.01 论文给出 LongMemEval-S 上 63.8% (gpt-4o-mini) / 71.2% (gpt-4o) ([arXiv:2501.13956](#))；DMR (Deep Memory Retrieval, **Zep 自己引入的基准**) 上 94.8% 对 MemGPT 的 93.4%——在自家引入的基准上对比对手，这个口径天然要打折 ([genalphai.com](#) 明确提示)。**第二层，第三方转述**：Bricks Technology 用 gpt-5.4 判官报出 LongMemEval 90.2% / LoCoMo 94.7%，但页面无运行日期 ([maximem.ai 汇编](#))。**第三层，争议数字**：Mem0 论文给 Zep 打 65.99%，Zep 自评 75.14% (并指控 Mem0 搞错说话人角色)，Mem0 去掉对抗题重跑得 58.44%——同一系统三个数横跨 17 个点，是第 5.5 节"判官敏感性"的最著名案例。AgentOS 审计独立复现 Zep 自报 71.2% 为 63.8% (同上来源)。**结论**：Zep 的工程价值在时序表示而非公开分数；评估它应跑官方 quickstart 的时间查询是否准确，而不是比谁的 LoCoMo 高。

研判

- 在"事实会变、需要可审计历史"的场景 (合规、客户档案、组织知识) 中，双时态图是当前最完整的表示方案，且这一能力"需要真正的定制工程才能复制" ([DEV Community](#))。
- Zep 宣称 p50 检索 104ms——如果属实，说明图检索延迟可以被工程压住。
- 时间索引 + 双时间戳是这一派的"护城河细节"：时间字段不是装饰，是有索引支撑的查询维度。

边界与辩证

- 顺序写入约束与高吞吐写场景 (每秒多条对话) 冲突，文献未见大规模并发写入的公开方案。
- 自托管 Graphiti = 自备 Neo4j/FalkorDB + 抽取模型 + 运维包装，第三方实测者承认"无法在不猜测 API 的情况下完成固定版本的复现"——运维门槛显著高于向量派。
- 时序失效依赖 LLM 抽时间戳，错误率无独立数据；无时间戳时用候选边 valid_at 作 invalid_at 是个脆弱的 fallback。
- Zep 商业云 only (Graphiti 自托管)，BYOC/云两种形态，数据边界需在采购前确认。
- 本节引用的分数横跨论文/转述/争议三层口径，且 Zep、Mem0 双方均维持竞争性记忆项目——任何一方的数字都不能单独采信。

第三部分 · 流派

第 10 章 精讲 · Letta 与 Generative Agents：自编辑与认知仿真

10.1 Letta (MemGPT)：把记忆当操作系统资源

Letta (前 MemGPT，源码实测归档的 V1 Python 服务器 0.16.8) 的核心思想：LLM 是 CPU，上下文窗口是 RAM，外部存储是磁盘，模型自己调工具在两层之间搬数据。实测机制：

- 核心记忆块**：Block (schemas/block.py) 有 value/limit/label/read_only 字段，persona 与 human 两块默认常驻系统提示，由 Memory.compile() 渲染成 XML 形态 (<memory_blocks><human>...</human></memory_blocks>)，块元信息 (chars_current/chars_limit) 随渲染注入；Anthropic provider 下还可切行号渲染模式辅助编辑；
- 自编辑工具族** (functions/function_sets/base.py, docstring 即 schema, 经 schema_generator.py 解析成 function JSON)：核心五件套 core_memory_append / core_memory_replace / memory_insert / memory_rethink / memory_finish_edits，升级版 memory_replace (校验 old/new 唯一出现，拒绝行号前缀) 与 memory_apply_patch (统一 diff) ——记忆编辑操作与代码编辑操作同构，"rethink"允许整块重写做大改压缩；
- 执行分发**：core_tool_executor.py 的 function_map 把壳函数映射到真实实现；编辑前先查 block.read_only (只读块抛错，双层防护)，改后 update_memory_if_changed 落库并重建系统消息；
- 三层记忆**：core (常驻块) → recall (全量消息表，Postgres FTS+向量+RRF 混合检索，实测返回 rrf_score/vector_rank/fts_rank 元数据) → archival (passage 归档 + 向量，先写 SQL 再按 provider 双写 Turbopuffer)；
- 心跳机制**：工具调用后强制再走一步 LLM (REQUEST_HEARTBEAT_PARAM, DEFAULT_MAX_STEPS=50)，形成"调工具→看结果→再决定"的自编辑循环；
- 元信息块**：<memory_metadata> 注入"recall 有 N 条、archival 有 M 条、可用 tags 有哪些"，**但不放内容**——内容必须靠工具显式换入；
- git 记忆仓**：块可序列化为 Markdown (YAML frontmatter + body)，落 ~/.letta/memfs/{org}/{agent}/repo.git/ ——记忆版本化到 git，memory 统一工具支持 create/str_replace/insert/delete/rename。2026 年 Letta 把这条线发扬为 git 化上下文仓库，Agent

用普通文件工具编辑 ([Particula Tech](#))。

注意：V1 Python 仓为归档状态，活跃开发已迁至 TypeScript 仓 github.com/letta-ai/letta-code (README 明示)。block_history 表实测支持 checkpoint/undo/redo ——记忆编辑可回滚。

三层的成本结构值得算清：**core 常驻 = 每 token 每轮都在计费**，所以块内容是记忆里最贵的部分；recall 混合检索有索引支撑但每次请求都要付 FTS+向量+RRF 的延迟；archival 便宜但召回率最低。MemGPT 论文的设计洞察正在于此——用“块的稀缺性”逼模型学会分级：什么配进 core（贵而常在）、什么该 archive（便宜但要检索）。这实际上是把经济学约束做进了架构。另按厂商口径，“Letta Filesystem”在 LoCoMo 上报 74.0% (gpt-4o-mini, 2025.08, [maximem.ai 汇编](#)) ——在自编辑派里这个数字反而说明：自主管理的记忆精度上限目前低于精心设计的检索派，自编辑买的是可控性不是精度。

与⓪派的本体区别：Mem0/Zep 是“服务替 Agent 记”，记忆生命周期归系统策略；Letta 是“Agent 替自己记”，记忆生命周期归模型决策。前者的质量上限是抽取/检索算法，后者的质量上限是模型使用记忆工具的能力。

10.2 Generative Agents：一切设计的认知科学母本

斯坦福 Smallville (joonspk-research/generative_agents, 2023-08 冻结) 贡献了四个被反复引用的设计，全部经源码逐行实测确认：

- **记忆流** (associative_memory.py, 文件头注释自证“this class is the Memory Stream module in the generative agents paper”)：ConceptNode 有 type (event/chat/thought)、poignancy (1-10 重要性)、last_accessed、**filling** (thought 指向其归纳依据的节点 id 列表)、depth (抽象层级：event=0, thought=1+max(证据 depth))。落盘纯 JSON 三文件 (nodes.json/kw_strength.json/embeddings.json)，无向量库；
- **三因子检索** (retrieve.py 的 new_retrieve)： $0.5 \times \text{recency} + 3 \times \text{importance} + 2 \times \text{relevance}$ (增益向量 gw=[0.5,3,2] 硬编码，各因子 min-max 归一化后加权，取前 30，命中后刷新 last_accessed)。relevance 是 O(N) 手写余弦——无 ANN 索引；
- **反思机制** (reflect.py)：累计重要性预算 (importance_trigger_curr 初值 150，每事件扣减其 poignancy) 花光即触发：LLM 提出 3 个焦点问题 → 三因子检索 → LLM 生成 5 条带证据编号的洞见 → thought 写回 (expiration=+30 天)。这是“何时反思”的第一个量化触发器；
- **认知闭环** (persona.py:185-231 实测主循环)：perceive → retrieve → plan → reflect → execute。每日规划用 new_retrieve 拉取反思产物修订 scratch.currently (当前状态)，当日计划本身也以 poignancy=5 写回记忆流——形成“经历 → 记忆 → 反思 → 身份/计划 → 行为”的完整闭环。

poignancy 打分本身就是个可复用的设计：不同行为类别有不同分值锚点 (对他人重要事件高分、闲聊低分)，LLM 只需对 1-10 校准。这让“重要性”成为可积累的量 (预算扣减) 而非每轮重算的布尔量。它证明了“检索打分 + 周期性抽象”足以让行为涌现出连贯人格——今天所有“记忆重要度评分 + 定期巩固”的设计都能在这里找到原型。工程复刻时注意两个实测坑：recency 因子方向与论文文字相反 (nodes 按 last_accessed 升序、decay 指数递减，归一化后最久未访问者得分最高，复算确认)；记忆落盘是纯 JSON 无向量库，规模上限极低。

研判

- 自编辑派适合 Agent 生命周期内聚、任务边界清晰的场景 (编码 Agent、个人助理)；“记忆块 + 文件工具”的模式已被 Claude Code 的 CLAUDE.md/MEMORY.md 实践证明可行。
- Generative Agents 的价值是设计语言而非代码——记忆流/三因子/反思/预算触发是公共思想财产，CrewAI 的复合打分就是它的现代化身。
- git 化记忆仓是自编辑派 2026 年最重要的演化方向：记忆编辑获得版本控制、diff 审查与回滚能力，与⓪文件派在“可审计”上合流。

边界与辩证

- 自编辑派强依赖模型能力：弱模型会滥用/漏用记忆工具，且每次自编辑消耗额外步数与 token；多服务共享中立的记忆 API 时反而不自然 (jangwook_kim 实测观点)。
- Letta 字符数上限 ≠ token 预算，limit 与实际 token 占用有偏差 (实测记录)。
- V1 归档仓内部分新工具 (memory/memory_apply_patch) 在 function_sets 里是 NotImplementedError 壳，真实逻辑全在 executor——读代码时注意壳与实现的分离。
- 认知仿真派未经工程化验证，直接用于生产系统属于误用。
- LoCoMo 74.0% 为厂商自报且用 gpt-4o-mini 判官，与第 14 章的复现争议数字不可直接比较。

第 11 章 精讲 · HippoRAG / Cognee / A-MEM：三个不同方向的探索

11.1 HippoRAG2：神经-图派的严谨样本

OSU-NLP-Group/HippoRAG（main 分支即 HippoRAG2）用 igraph 建图（默认无向、pickle 持久化 + FileLock + os.replace 原子写），节点三类（实体/篇章，事实只存向量仓不建节点），边三类（fact 边带 fact_source_counts 出处计数、passage 边权重恒 1.0、同义边 KNN 阈值 0.8 每实体 ≤100 条）。写入（index(), HippoRAG.py:488 实测）：切块（overlap=128）→ chunk 入向量仓 → 按 md5 增量去重只对新 chunk 跑 OpenIE → 两阶段抽取（ThreadPoolExecutor 8 并发：先全量 NER，再并行三元组抽取，prompt 要求"resolve pronouns"）→ 实体/三元组分别嵌入入库 → fact 边（统计 fact_source_counts）→ passage 边 → 同义边（KNN 2047 近邻）→ 合图。检索链路见第 5 章样本二。

HippoRAG2 相对 v1 的关键取舍是弃 GNN 改纯稠密 + PPR（v1 用 fine-tuned RoBERTa 做图上节点打分）：离线资源显著低于 GraphRAG（不做社区摘要）与 RAPTOR，同时保留多跳能力——它是 GraphRAG 家族的"轻量对照系"。工程上还有两个被低估的设计：出处感知删除——delete() 按 chunk 删事实边出处计数，计数归零才删边/节点（增量写删的安全姿势）；索引身份校验——index_manifest.json 记录模型/端点/归一化身份，不匹配即拒绝复用旧索引（换嵌入模型后旧索引静默错配是常见事故）。

生态位建议：HippoRAG2 的参数敏感度（damping/linking_top_k/同义阈值）决定了它不是"开箱即用"型系统——合理的用法是把它当多跳检索的研究基座，在自己的多跳题集上调准参后再上量；单跳为主、高频写入的对话记忆场景应直接排除。它的两个工程习惯（出处计数删除、索引身份校验）则适用于任何图派系统，值得整体移植。

它的价值不止于系统本身，更在于诚实记录了坑点（真实笔记文件实测归纳）：同义边阈值敏感（低则错连、高则孤岛）、compute_mdhash_id 直接 hash 原文导致"Amazon/Amazon Inc."分裂为两节点、PPR 参数敏感、增量写只能追加不能原地改、谓词不建节点（无法按关系类型过滤）、rerank 用 LLM 逐 query 调用在线成本不低。

11.2 Cognee：把"记忆化"做成可组合流水线

topoteretes/cognee 的公共 API 是 add / cognify / memify / search（以及 remember/improve/recall 语法糖）。它的独特贡献是把"记忆维护"拆成可按名组合的任务注册表（memify_task_registry.py 实测）：detect_entity_duplicates、merge_entity_duplicates、cross_connect_entities、apply_frequency_weights、apply_feedback_weights、index_data_points每个任务都是"extraction 读图产出 + enrichment 落库"两段式。六个富化阶段的实测细节：

阶段	机制（实测）
A 三元组向量化	分页拉全部三元组，按 start→rel→end 拼可嵌入文本，id = generate_node_id(start+rel+end)，写 Triplet_text collection
B 实体对齐合并	名字向量 0.85 阈值 + 归一化名相等的并查集聚簇（分块扫描，每节点 top 10 近邻）；度数最高者为 canonical；dry_run 支持
C 跨实体连边	邻居 IDF 加权 Jaccard（阈值 0.2）+ 名字向量相似（0.5）两路候选 → LLM 判关系 → confidence ≥0.7 成边，推断边带 feedback_weight=0.2 便于优先剪枝，每节点新边 ≤5
D 频次加权	会话 QA 用到的图元素 frequency_weight += 1.0，幂等标记防重复
E 反馈加权	1-5 分归一化后 EMA 更新：w' = w + 0.1(norm - w)，clip [0,1]
F 全局上下文索引	自底向上层级摘要树：每 20 个 item 聚一桶（LLM 摘要），逐层聚类至根；桶放置默认按向量距离（0.5 阈值），graph 策略实验级

检索侧（第 5 章 + cognee_graph 实测）：向量距离投影到内存子图后按三元组打分 score(edge) = val(node1) + val(node2) + val(edge)，元素分 = 原始距离 × (2 - importance_weight) 修正，再与 (1 - feedback_weight) 线性混合；缺失向量距离的元素用 triplet_distance_penalty=6.5 兜底罚分。这是六路线里工程化程度最高的"记忆治理"抽象。

阶段 C 有一处容易被误读的细节值得展开：推断边 (inferred edges) 与观测边是两类证据等级。观测边来自用户明说的事实，推断边来自 LLM 的关系猜测 (confidence ≥0.7 才入库) ——Cognee 用 inferred: True 元数据和更低初始权重 (0.2) 做区分。这个区分在

审计和被遗忘权场景（第 6.4 节）都会用到：删一条观测事实时，“由它推断出的边”是否连带删除，需要知道边是观测的还是推断的。自建图派系统应把 evidence_type 作为边的一等属性，而不是事后补的元数据。

11.3 A-MEM：Zettelkasten 卡片盒的教训

agiresearch/A-mem（NeurIPS 2025 论文配套，arXiv:2502.12110，arxiv.org/abs/2502.12110）把记忆做成 LLM 可自主增删改、互链的卡片节点：写入时找 5 个最近邻，LLM 决定 strengthen（加链接/改标签）或 update_neighbor（改写邻居元数据），演化达阈值触发整体巩固。理念激进：记忆是可变的（mutable），不是只追加的（append-only）。

但官方精简仓库的实测暴露了理念落地最真实的工程陷阱：① 链接存的是枚举序号而非 UUID（find_related_memories 返回 0..k-1，prompt 展示 "memory index:{i}"，LLM 返回的 suggested_connections 就是序号串），search_agentic 用 memories.get(link_id) 永远取不到——链接扩展在主路径上实际失效；② update_neighbor 按 list(self.memories.values()) 下标回写，dict 顺序一变就张冠李戴；③ evolution_history 字段存在但代码从不 append，改写不可追溯；④ consolidate 全量重建 Chroma，O(N) 且丢磁盘状态；⑤ 每次写入至少 1 次演化 LLM 调用且 temperature=0.7 非确定，成本随记忆数线性增长。教训：让 LLM“自主改写记忆”的前提，是身份引用系统本身可靠——UUID/稳定 id 是一切 agentic memory 的地基（Cognee 的确定性 id 与 Mem0 的整数映射是正面对照）。另注意：精简仓与论文实验仓（WujiangXu/AgenticMemory）有行为差异——论文原版有 1-10 显著性打分且 add_note 会调用分析，精简仓没有。

把 A-MEM 的五个陷阱升华为一张“自主改写记忆”的安全清单（适用于任何想让 LLM 修改自身记忆的系统）：① 引用必须是全局稳定 id，禁止位置/序号引用；② 每次改写必须 append 历史（who/when/why/before/after），缺历史等于不可回滚；③ 改写范围要有白名单（邻居的 context/tags 可改，正文/创建时间不可）；④ 温度设 0 或对关键改写做双次一致校验；⑤ 全量重建类巩固必须可增量、可中断、可恢复。这五条对应的事故模式在 A-MEM 里全都能找到一一对应的代码。

研判

- HippoRAG2 适合文档型多跳问答（知识库记忆），不适合高频对话记忆（写入成本不匹配，实测确认）。
- Cognee 的任务注册表是“记忆治理流水线化”的正确抽象方向，值得在设计自有系统时借鉴；其出处感知（fact_source_counts）与索引身份校验是图派落地的两个必抄细节。
- A-MEM 证明了 mutable memory 的理念可行，但生产落地需自建身份/溯源/回滚层——精简仓库恰恰缺这三样。
- 三个系统共同指向一个分工格局：HippoRAG2 管多跳检索研究、Cognee 管记忆治理流水线、A-MEM 提供自主演化的问题清单——三者是互补的素材库而非竞品。

边界与辩证

- HippoRAG 的 OpenIE 两阶段建库在大语料上依赖离线批处理（offline 模式分两段跑，先 vLLM 离线抽好再建图），在线增量体验一般。
- Cognee 的 LLM 推断边（confidence 阈值）会引入不可验证的“幻觉边”，审计时需区分观测边与推断边（其 inferred: True 元数据提供了区分依据）。
- Cognee 的 feedback_influence 默认 0、graph 桶策略实验级——文档里的高级特性与默认路径之间有距离。
- A-MEM 论文结果复现需用实验仓而非精简仓。
- 三者均为研究/早期工程项目，生产采用前需评估维护活跃度与社区规模。

第三部分 · 流派

第 12 章 产品级记忆：从 Dreaming 到 AGENTS.md

研究流派之外，2025-2026 年的消费级与开发工具级记忆已经形成另一个战场。六组事实：

12.1 ChatGPT：Dreaming 与“被管理的遗忘”

ChatGPT 记忆从 2024 年 2 月的显式“saved memories”起步，2025 年 4 月扩展为可参照全部历史对话；2026 年 6 月 4 日上线 Dreaming 更新：后台跨会话合成记忆、随时间修订时效性事实，官方报告事实召回从 2024 式记忆的 41.5% 升至 82.8%（另一口径：V3 使 67.9%→82.8%）。同期的架构信号：有状态 Assistants API 于 2026 年 8 月 26 日弃维，转向无状态 Responses API + 持久化 Conversations API——状态管理从模型 API 层剥离，记忆作为独立基础服务。出处：taskade.com、o-mega.ai。

Dreaming 的产品化路径有两条值得借鉴的克制设计（对照第 6.2 节的 MemOS Dream）：其一，**修订而非覆盖**——"你七月要去新加坡"修订为"你 2026 年 7 月去了新加坡"，时效性事实随时间改写时保留可理解的演变叙事，而不是静默替换；其二，**用户可见的记忆面板**——合成与修订结果可被用户查看和纠正。这两条恰是"后台改写黑盒"担忧（见本章辩证栏）的产品级缓解。

12.2 Anthropic: memory tool + 上下文编辑的开发者路线

2025 年 9 月 29 日 Anthropic 同时发布两个原语：**memory tool**——模型通过工具调用在开发者控制的 /memories 目录做 CRUD，存储后端完全由开发者掌控，"operates entirely client-side through tool calls"；**context editing**——上下文将满时服务端自动清除过期的工具调用与结果。官方报告两者结合带来 39% 性能提升、100 轮搜索任务 token 节省 84%（context editing 单独为 29%）。Claude Code 则用最朴素的文件方案：CLAUDE.md（项目根 + 全局 ~/.claude/，会话开始载入系统提示）+ MEMORY.md 索引（前 200 行自动载入）+ auto memory（ ~/.claude/projects/<path>/memory/ 下按交互模式自动积累笔记）+ 上下文压缩边界上记忆文件存活。出处：[o-mega.ai](#)、[crabtalk.ai](#)。

Anthropic 路线的独特立场是**"记忆的存储权归开发者"**：memory tool 的后端是开发者自己的文件系统/数据库，模型只是通过工具读写。这与 Mem0/Zep 的"记忆托管给第三方"形成治理光谱的两端——对合规敏感场景（医疗、金融、政务），"记忆数据不出域"可能比检索精度更重要。选型时这是一个经常被遗漏的一级问题：记忆存谁的云？

12.3 其他平台的快照

平台	机制（公开资料口径）
Google Vertex AI Memory Bank	Gemini 异步抽取/整合/消解矛盾的用户事实，按相似度检索，约 \$0.25/千条存储；与 ADK/LangGraph/CrewAI 集成
Gemini 消费版	2025.08 自动记忆 + 隐私控制；2026.01 Gemini 3 Personalization；2026.03 记忆导入（5GB ZIP）；2026.04 Gemini Notebooks（工作区内全量可读的结构化长上下文）
AWS Bedrock AgentCore	语义+情景记忆托管，框架无关（2026 综述口径）；Mem0 成为 AWS Agent SDK 独家记忆供应商（厂商披露）
Microsoft Foundry Memory / Weaviate Engram / Cloudflare Agent Memory	2026 年中发布的托管记忆层，架构与 Mem0 收敛（异步抽取+检索优先）
新兴托管层（第三方汇编）	Hindsight（LME-S 94.6 / LoCoMo 92.0 自报，gemini 判官）、Evermind EverOS（LoCoMo 93.05 / LME 83.00，GPT-4.1-mini 判官，arXiv:2601.02163）、SuperMemory（95% Recall@15，检索口径非准确率）——数字均为各自自报，汇编自 maximem.ai

综合 [o-mega.ai](#)、[chatforest.com](#)、[dev.to](#)、[maximem.ai](#)。

生产架构上还有一个被广泛使用、但很少写进记忆系统文档的**双 layer 模式**：会话级热状态（最近上下文、工作缓冲）放 Redis/DragonflyDB 类内存缓存，跨会话语义记忆放 Pinecone/Qdrant/Chroma 类向量库——前者是工作记忆层，后者承载情景与语义记忆（[clawdbot2.in](#) 归纳的生产常见模式）。它与第 3.3 节的五层分层兼容：缓存层对应"会话缓存"，向量库对应"语义层"，中间的情景层往往被忽略——而丢失情景层（原始消息）的系统，恰恰是第 4.1 节实证里最吃亏的形态。

12.4 Markdown 文件的胜利

2026 年最"反直觉"的事实：**最流行的 Agent 记忆格式是纯 Markdown**。AGENTS.md（Linux 基金会下 Agentic AI Foundation 维护，据称 6 万+ 开源项目使用）与 CLAUDE.md 是"可以放进 PR 里 review 的记忆"；CrabTalk 对五产品的调研给出量化对比：Claude Code 可审计性 9/10 但机器可检索性仅 2/10，Cursor/Windsurf（.cursorsrules）类似，OpenClaw 正相反（9 vs 4，最大化机器检索），ChatGPT 两者皆低（开发者视角）。可审计与可检索是这个光谱的两端，**目前没有产品同时占优**。出处：[taskade.com](#)、[crabtalk.ai](#)。生态补位也发生在文件侧：claude-mem 加语义压缩、memsearch 做 markdown 索引、Basic Memory 提供 MCP 上下文——文件派的"可检索性短板"正由第三方工具补齐。

12.5 Basic Memory: 文件派与 MCP 的合流

basicmachines-co/basic-memory（源码实测，AGPL-3.0，~3.3k star）是文件派最完整的工程实现：Markdown 文件是唯一事实源（source of truth），SQLite/Postgres 只是派生投影（DB-first 写入后异步物化落盘，NoteContent.file_write_status 状态机 pending/writing/synced/failed/external_change_detected 跟踪同步）。笔记格式三要素实测：**Observation**（ - [category] content

#tag (context)，解析正则 `^[[([^\[\]]+)]\s+(.+)`)、**Relation** (显式 - type [[Target]] (ctx)；散文里的裸 wikilink 落为 links_to；**前向引用允许**——target 不存在先挂 NULL，创建时回填，删除用 ON DELETE SET NULL 保留断链可见)、**permalink** (稳定 id，文件移动不变)。

五个实测的工程细节值得记录：① frontmatter malformed (两个 --- 之间非法 YAML) 时"索引但绝不回写"——不知道哪几个字节是元数据就不碰；② 相对时间不可重放——@occurred:yesterday 用两个相距极远的基准时间探针复算，读数不一致即拒绝，保证"从文件字节可复现"；③ 误判防御——任务列表 - []、时间码 [00:00:11]、Obsidian callout 都会撞 Observation 形状，解析层逐一排除；④ 对 Claude Code 的桥接用 hooks——session_start 注入简报、pre_compact 压缩前做检查点；⑤ 写冲突防御——默认不覆盖，冲突返回 NoteAlreadyExists，移动留"墓碑"记录区分残留与副本。适用边界 (源码实测归纳)：个人/单用户 coding agent 零运维首选；多人共享、需要自动抽取与冲突消解时让位于 Mem0/Zep。

它的"文件为源、DB 为投影"架构有一个更普遍的名字：**读模型/写模型分离 (CQRS)**。文件是写侧权威 (人可编辑、git 可版本化)，SQLite 索引是读侧投影 (机器可查询、可随时重建)。这个模式的深层好处是**投影可丢弃**——换嵌入模型、换索引结构时只需重建投影，权威数据零迁移。任何"文件派 + 检索需求"的场景都值得套用这个模板，而不必绑定 Basic Memory 本身。

12.6 编码 Agent 的"文件即记忆"谱系

把源码实测的 coding agent 记忆放一起，能看到"文件派"在 2026 年的完整谱系：

系统	跨会话记忆	会话内记忆	压缩机制
Claude Code	CLAUDE.md + MEMORY.md (前 200 行) + auto memory 目录	窗口 + compaction (记忆文件跨压缩边界存活)	自动压缩
OpenHands	.openhands/memory/ 笔记 (默认关) + PLAN.md	task_list (内存) + 事件流	condenser (视图裁剪，独立计费)
Cursor / Windsurf	.cursorsrules 等规则文件	窗口	未公开细节
Basic Memory (外挂)	Markdown 知识库 + MCP	memory:// 图导航	pre_compact 检查点

12.7 治理面：消费级记忆正在被监管重塑

消费级记忆的下一阶段竞争轴是治理。EU AI Act 透明度要求 2026 年 8 月全面生效后，用户对 Agent 记忆的三项权利 (知情、复核更正、请求删除) 进入合规清单；领先平台已在把治理做成产品能力而非事后补丁——多 scope 记忆 API (user/session/agent 分层) 使定向删除成为可能，append-only 审计日志使每次记忆操作可追溯 (clawdbot2.in)。企业场景还有第二重治理动机：**陈旧记忆本身就是事故源** (过时的收货地址、已离职成员的权限约定)，记忆系统需要更新与失效机制，而不仅是存储与检索 (同上来源)。给产品团队的检查单：记忆面板 (用户可见全部记忆) → 逐条编辑与删除 → 导出 (可迁移性) → 删除传播 (派生摘要/图边一并清理，见 6.4) → 审计日志。五项里缺前三项，2026 年后的面向欧盟用户产品已有合规风险。

研判

- 消费级记忆的竞争焦点已从"记不记得住"转向**治理** (用户可见、可改、可迁移) 与**遗忘调度** (Dreaming 类后台整理)。
- 开发工具级记忆收敛于"文件 + 索引"双形态：文件保人可审计，索引保机器可检索——Basic Memory 的"文件为源、DB 为投影"是可复制的架构模板。
- coding agent 是"文件即记忆"的最佳试验场：工作目录本身就是可寻址存储，检索退化为 ls/grep。
- "记忆存谁的云"是选型的一级问题：Anthropic 的开发者持有路线与 Mem0/Zep 的托管路线在合规光谱上位于两端。

边界与辩证

- Dreaming 类机制的"后台改写"对用户是黑盒：改错了什么、依据什么，消费者无法审计——EU AI Act 2026 年 8 月全面生效的透明度要求会倒逼改变 (clawdbot2.in)。
- Markdown 记忆的规模化上限明显：CrabTalk 调研中文件派的可检索性普遍 ≤2/10；"CLAUDE.md 前 200 行"本质是手工分页。
- Basic Memory 的 AGPL-3.0 对闭源商用有传染性 (源码实测确认)，选型前先过法务。
- OpenHands 的 Python 实现本体在另一个仓库 (software-agent-sdk)，可获取的仅有 TS 镜像——公开资料口径与一手代码之间要区分。
- 新兴托管层的自报分数 (Hindsight/EverOS 等) 未经独立复现，且汇编方 Maximem 自身也是竞争性记忆厂商——汇编的"第三方性"有限。

- 监管解读（三项权利的适用范围）以法规文本与法律意见为准，本节是工程视角的转述。

第三部分 · 延伸

第 13 章 延伸 · 企业上下文：组织级记忆与 Agent 记忆的会师

写完六条流派，本书可以回答"一个 Agent 怎么记住"，但企业在落地时几乎必然撞上另一个问题："我们的 Agent 为什么不了解我们公司？"——它记住了用户的偏好，却不知道公司上季度的定价策略；它记住了对话历史，却不认识 CRM 里那个客户。这一章讲的"企业上下文"（Enterprise Context / Context Layer），就是 2026 年行业对这个问题给出的答案。它和 Agent 记忆同根同源——都是在"有限窗口 × 无限信息"的约束下解决"正确时机放入正确信息"——但分属两个所有权层级。理解它们的关系，是从"给一个 Agent 配记忆"升级到"给一个组织配认知基础设施"的关键一步。

13.1 是啥：定义与边界

定义（本书口径）：企业上下文是组织级、被治理、供所有 Agent 与应用共享读取的背景信息层——把散落在数仓、CRM、文档、Slack、代码仓库里的知识，织成一个带语义定义、业务规则、权限映射和来源溯源的语义模型，让 Agent 读到的不是原始数据，而是"企业认可的含义"。Widgetstack 给的朴素版本是：企业上下文就是"Agent 在你组织里干有用活所需的业务专属知识——你的客户、定价、决策、政策、当前优先级"。没有它，Agent 只能从通用训练数据出发推理，产出"听起来合理、但对你的处境是错的"的答案。

最系统的表述来自 Palantir 的 Ontology 论述：传统数据架构只存"数据"，不存"决策"，而任何一个运营决策都由四要素构成——

要素	内容	企业上下文里的形态	Agent 记忆里的对应物
数据	做决策所依据的信息	对象/属性/链接（ERP、IoT、文档统一成语义对象）	情景记忆（对话事实）
逻辑	评估决策的启发式与计算过程	业务规则、指标定义、函数（挂在对象类型上）	程序性记忆（怎么做事）
动作	决策的编排与执行	Action 类型（带校验与审批的可执行操作）	工具调用历史
安全	决策合规的保证	行级/列级权限、目的约束、审计	scope 隔离（第 3 章）

这张表已经暗示了本章结论的一半：企业上下文四要素在 Agent 记忆里都有低配对应物——两者是同一套认知需求在"组织层"与"个体层"的两个实例。

边界澄清——它不是什么：企业上下文不等于 RAG（RAG 是一种把上下文送进窗口的传输手段，上下文层是生产并治理这些内容的层）；不等于数据目录（目录存"元数据"，上下文层把元数据、业务规则、权限、时效绑成一个可执行整体）；也不等于知识图谱（图是可选的表示形式之一，Graphiti 那种时序图可以充当上下文层的存储引擎）。Pebblous 的区分最干净：静态语义层只回答"字段是什么意思"，上下文层额外带实时更新、权限映射与关系推理。

13.2 为啥现在爆发：三个事实

企业上下文不是新概念（本体工程、主数据管理做了二十年），它在 2026 年成为独立议题，靠的是三个可验证的事实：

- 事实一：模型同质化，上下文成为差异化资产。当所有企业都能调用能力相近的前沿模型时，"你的 Agent 比别家懂你的业务"的唯一来源就是上下文。Atlan 的表述：客服 Agent 在两家银行表现不同，不是因为模型，而是因为各自编码了不同的规范与决策规则。这与本书第 1 章的公理互为镜像：Transformer 无状态 → 个体智能靠记忆补 → 组织智能靠上下文补。
- 事实二：MCP 解决了"搬运"，并把"生产"的欠账暴露出来。MCP（Model Context Protocol）2024 年 11 月由 Anthropic 发布，2025 年 12 月捐赠给 Linux 基金会旗下 Agentic AI Foundation；据 Context Chronicle 与 Pebblous 的汇编，其月 SDK 下载量在 11 个月内从 10 万涨到 9700 万（约 970 倍），公共服务器超 1 万个，ChatGPT、Cursor、Gemini、Copilot、VS Code 均已支持，Forrester 预测 2026 年内 30% 企业应用厂商将自带 MCP server。但协议只负责把上下文从 A 搬到 B，不负责上下文本身是对的、新的、有权跑的——Gartner 分析师在 2026 Data & Analytics Summit 上的预测被广泛引用：仅依赖 MCP 而缺乏一致语义层的 agentic 分析项目，60% 将在 2028 年前失败（转引自 Atlan）。一句话：管道标准化了，水管里没水。
- 事实三：效果可量化。Atlan AI Labs 用 174 个查询、522 次评测（13 张表、94 列）对比"带治理语义元数据 vs 不带"的 AI 生成 SQL：加入上下文后相对准确率提升 38%（p<0.0001），客户工作坊自报最高 5 倍查询准确率改善（atlan.com，厂商口径、方法已

公开，注意与第 14 章的"自报基准"纪律保持一致)。这给了 CDO 一个能过会的数字。

13.3 怎么做：一条被验证的工程路径

综合 Atlan、Pebblous、Widgetstack 三家的实践叙述，2026 年收敛出的做法高度一致，可概括为五步（每步都附"最常犯的错")：

步骤	做什么	最常犯的错
1. 从已有资产起步	先挖语义层里的指标定义、数据目录里的表元数据、数仓查询历史——这些是被验证过的含义，比绿地写文档又快又准	找咨询团队从零写"企业知识文档"，写完即过时
2. 补齐长尾	业务术语表、历史 MDM 遗产的本体、操作手册、管道代码、从没进过数仓的口传知识	追求"完整覆盖"而不是"按错误率排序补齐"
3. AI 提议、人审批	上下文 Agent 批量建议同义词、join 关系、过时定义与描述草稿，领域专家只做认证放行	全人工（产能不够）或全自动（错误量产）
4. 按业务影响排优先级	追踪"哪些上下文缺口正在制造真实 Agent 错误"，按业务损失排序修	按目录字母序补元数据
5. 当系统运营，不当项目结项	业务在变、定义在演进、新系统上线——上下文层需要持续的"context engineering"运营职能	结项后无人维护，一年后沦为"昂贵的过期文档"

在交付形态上，2026 年的关键词是 **Context Repo（上下文仓库）**：[Atlan 的模型无关指南](#)提出五条 pass/fail 级的验收标准——①上下文以版本化、有界、开放格式的包发布；②任何模型经开放协议（MCP）读取，无厂商专有集成；③修正一处、所有接入模型受益（不是各平台各学各的）；④上下文物理可导出（"那个存在我们专有格式里"是红旗）；⑤换模型不动上下文。这五条本质上是本书反复出现的"反锁定"纪律在上下文层的重演——与第 12 章 Basic Memory 用 Markdown+git 保证"你的记忆可带走"是同一个设计价值观的组织级版本。

两条生产红线：一是**权限镜像**——上下文层必须继承源系统的权限模型，Slack 私有频道的内容只能被该频道成员的 Agent 会话检索到（Widgetstack）；二是**强制溯源**——每个答案带回源文档引用，"AI 说了算"不构成审计链（这一点与第 2 章"好记忆多维评价"的第五维完全同构）。

13.4 谁在做：三种代表形态

形态	代表	核心主张	与 Agent 记忆的关系
平台内建型	Palantir Ontology（Foundry/AIP）、Snowflake、Databricks 各自的语义层	把本体做成"决策的表示"：数据+逻辑+动作+安全四合一，2026 年 Foundry MCP 与 Ontology MCP 已 GA，对象类型/Action 经 OAuth 权限暴露为 Agent 工具	官方明确说 Ontology 捕获的"决策数据"（决策上下文、被评估的选项、下游后果）用于"持续精炼各类 agentic memory（工作/情景/语义/程序）"——平台把上下文当作记忆的地基（ Palantir 官方博客 、 Foundry LLM 摘要文档 ）
跨平台中立层	Atlan 等"上下文层"厂商	多数企业跑 3-5 个数据平台，平台内建上下文对"平台外 80% 的业务"是盲的；跨平台身份解析、统一权限、端到端列级血缘才是真需求	Context Repo 作为共享基础设施，Agent 记忆框架各自负责"私有笔记本"，两者分层（同上 Atlan）
Agent 厂商自建记忆层	Anthropic Managed Agents 记忆（2026-04 公测）	记忆存为文件系统上的结构化文件，可经 API 导出/编辑/管理；2026-05 的 Dreaming 把"离线巩固"做成生产特性，支持"自动更新或暂存待人工审批"两种模式	案例数字（厂商口径）：Rakuten 长时任务 Agent 首过错误降 97%、成本降 27%；Netflix 跨开发者会话保留多人纠正上下文；Wisedocs 靠跨会话记忆识别重复文档问题（ Context Chronicle 汇编 ，未独立审计）

注意第三种形态与本章主题的张力：Agent 厂商的记忆层做的是"个体记忆的产品化"，而 Rakuten 那 97% 的错误下降，靠的恰恰是"Agent 把教训写成结构化文件、其他 Agent 可读"——这已经是记忆向组织上下文升华的雏形（共享文件系统扮演了 mini Context Repo）。三条路线正在互相走进对方的领地。

13.5 与 Agent 记忆的关联：同根、分层、双向流动

这是本章的核心小节。两者同根——都源于第 1 章的根本矛盾（无状态模型 × 持续变化的任务）；不同层——分属个体与组织的认知所有权。六维对照：

维度	Agent 记忆	企业上下文
归属	Agent 实例 / 用户（第 3 章 scope 体系的最小粒度）	组织（enterprise scope 是 scope 体系的最大粒度）
写入方式	运行中自动抽取（LLM 增量写，见第 4 章）	人工治理 + AI 提议 + 专家审批（curated pipeline）
信任模型	经验性的 ：Agent 亲身经历，可能含噪声、幻觉、注入（第 15 章记忆投毒）	权威性的 ：经认证的组织事实，错误来源是"过时"而非"编造"
生命周期	随会话/实例生长，靠遗忘与巩固控制（第 6 章）	随组织演进，靠版本化与运营流程保鲜（13.3 步骤 5）
治理对象	个人删除权、可见性（EU AI Act 场景，第 6 章）	访问控制镜像、审计链、合规（"AI 说了算"不构成审计）
典型失效	漂移、投毒、冲突未消解、静默降级	过时定义、碎片化盲区、平台锁定、语义层缺失

认知科学的类比可以帮助定位：企业上下文之于组织，相当于**语义记忆 + 文化**（被编码、被共享、可教学的陈述性知识）；Agent 记忆之于个体，相当于**情景记忆 + 程序性记忆**（亲历的、含技能的）。一个人可以读遍公司 wiki 却毫无实战经验（有上下文无记忆），也可能身经百战却说不出公司定义（有记忆无上下文）——健全的组织认知需要两者。

工程上两者有两条**双向流动**管道，这也是 2026 年架构演进最活跃的方向：

- **上行：记忆升华（memory → context）**。Agent 的经验性发现，经人工审批沉淀为组织资产。已落地的形态：Anthropic Dreaming 的"暂存待审批"模式（Agent 提议巩固结果，人确认后落入共享记忆文件）；Palantir 的"决策数据"设计（边缘用户与 Agent 做决策时产生的上下文、选项评估、后果被 Ontology 捕获，成为新的组织知识）；第 12 章 Basic Memory 的 git 化 Markdown 仓若在团队内共享并加审批流，即是最小可用的升华管线。设计要点与第 6 章巩固机制完全同构：候选（Agent 提议）→ 审查（人或规则）→ 合并（版本化落库）。
- **下行：上下文做记忆种子（context → memory）**。新 Agent 冷启动时最大的痛点是"零记忆"（第 15 章实例延续问题）：以企业上下文中与该 Agent 职责相关的切片作为初始记忆/系统提示素材，等于"给新员工发入职手册"。Letta 的 core memory block、Claude Code 的 CLAUDE.md，本质都是"上下文在个体记忆中的常驻投影"。工程含义：上下文层的**切分粒度与 scope 设计必须对齐**（第 3 章），否则下行注入会发生权限越界——把 A 团队价格策略注进服务 B 团队客户的 Agent，是权限事故而不是检索失误。

企业上下文 × Agent 记忆：分层与双向流动



同根：无状态模型 × 持续任务 | 分层：组织语义记忆 vs 个体情景/程序记忆 | 通道：MCP 等开放协议

13.6 挑战：六个真实的坑

挑战	实质	证据/出处
碎片化盲区	企业平均跑 3–5 个数据平台；平台内建上下文对平台外业务是盲的，Agent "在自己平台内看得见、跨平台全瞎"	Atlan
语义层缺位	有管道没内容：Gartner 预测仅靠 MCP 缺语义层的 agentic 项目 60% 在 2028 年前失败	同上（峰会引述）
治理成熟度悖论	能画清晰上下文图本身就是数据治理成熟的信号；图难画的公司，上上下文层反而倒逼治理补课——"买工具"实质是"补组织能力"	Pebblous
供应商锁定	平台厂商有商业动机把上下文层做成"整合度越高越好用"；Atlan 创始人的建议是直接问三件事：上下文是否开放、是否可互操作、离开时会发生什么	Atlan
权限与合规继承	上下文层聚合了原本分权存放的信息，聚合本身就改变攻击面；行级权限、目的约束、审计链必须随上下文走，而不是在 Agent 端重造	Palantir 官方文档的 OAuth/权限模型；第 6 章 EU AI Act 管线
预算错配	企业 AI 预算集中在计算与编排，"context 是 2026 预算里缺失的科目"——管道买了、内容没人养	Pebblous

13.7 场景与建议

三个代表性场景，展示两层如何协作（第三个贴近本书读者的典型行业）：

场景	企业上下文提供	Agent 记忆提供	协作点
客户支持	产品目录、定价政策、SLA 条款、历史承诺（CRM 对象）	该客户的沟通偏好、历史投诉、上次未决问题	Agent 用上下文查"我们能承诺什么"，用记忆查"我们对他承诺过什么"——两者不一致时以治理层为准并触发上报
数据分析	指标口径、表血缘、认证 join 关系（语义层）	该分析师的常用口径偏好、历史纠缠记录	Atlan 基准的 +38% 即来自此场景；个人记忆负责"这位 CFO 看的 GMV 不含退款"，上下文负责"GMV 官方定义"
机务维修（航空）	机型构型库、工卡模板、AMM/IPC 手册版本、放行规则（本体对象 + Action）	特定尾号飞机的历史故障与处置、该工程师的排故偏好	Agent 查上下文知道"B737-800 件号 X 的标准工卡流程"，查记忆知道"B-1234 这台 IDG 上次更换后的振动趋势"——预测性维护正是两层信号的交点

给实践者的五条起步建议（综合三家的共识做法，按性价比排序）：

- 先做错误归因，再选源：找出当前 Agent 答错的 80% 问题来自哪 3–4 个信息源，只接它们（Widgetstack）；
- 从已有元数据起步，不写绿地文档（13.3 步骤 1）；
- 上线前验证两件事：权限镜像是否成立（用低权限账号试越界）、每个答案是否带源引用（溯源是硬门槛不是加分项）；
- 给"上行管道"留好接口：即便第一阶段只做只读上下文，也要让 Agent 记忆的写入口带 proposal/审批状态字段——后补这个字段等于改表结构；
- 用第 14 章的纪律审视厂商数字：+38% 这类基准要看方法是否公开、评测集是否独立，"厂商口径"四个字永远默认标注。

口径与来源汇总：定义与工作流 [atlan.com/know/context-layer-for-ai-agents](#) 与 [atlan.com（模型无关指南）](#)；决策四要素与记忆反哺 [blog.palantir.com/connecting-agents-to-decisions](#) 与 [palantir.com Foundry 平台摘要（2026-07 更新）](#)；MCP 生态数据与 Anthropic 记忆案例 [contextengine.tech](#)；治理悖论与预算视角 [blog.pebblous.ai](#)；落地 FAQ 与权限镜像 [widgetstack.ai](#)。注意：Atlan、Pebblous、Widgetstack 均为上下文层相关厂商，其论述自带立场，数字均为厂商口径。

研判

- 企业上下文与 Agent 记忆是同一根本矛盾（无状态模型 × 持续任务）在组织层与个体层的两个解，2026 年的证据（MCP 标准化、Gartner 语义层预测、Atlan 量化基准、Palantir 记忆反哺设计）表明两者正在合流为"两层认知基础设施"，而非竞争方案。
- 对多数企业，正确的建设顺序是上下文层先行、记忆层随后：上下文层是只读的、权威的、可治理的，先建它风险最小；记忆层的写路径（投毒、漂移）风险更高，适合在治理框架就绪后打开。
- "上行升华"是未来 12–24 个月最有架构价值的方向：谁能把 Agent 经验安全地变成组织资产（审批流 + 版本化 + 溯源），谁就同时解决了记忆的合规问题和上下文的保鲜问题。
- 选型优先用 13.3 的五条 pass/fail 与 13.6 的锁定三问，而不是厂商的能力清单——本书第 14 章的"<10 点差异不构成证据"纪律在上下文层同样适用。

边界与辩证

- 本章主要证据来源（Atlan、Pebblous、Widgetstack）本身就是上下文层赛道的玩家，叙事自带"这很重要"的动机；Gartner 60% 失败预测与 Forrester 30% 预测均为转述，未见原始报告全文——把它们当"方向信号"而非"精确概率"。
- "企业上下文"没有公认定义：Palantir 说 Ontology"不是语义层"，Atlan 说上下文层高于语义层， vendors 各自定义有利于自己的边界——本章 13.1 的定义是本书综合口径，采购时应要求厂商给出书面定义再对齐。
- Rakuten 97%、+38% 等数字均为厂商口径、未经独立审计，且这些是"有上下文 vs 无上下文"的对照，不能外推为"厂商 A 比厂商 B 好 38%"。
- 上下文层不是银弹：它治"不知道"，不治"推理错"。第 14 章的判官缺陷、第 15 章的开放问题在两层都存在；把上下文层建好之后，Agent 依然会用错误的推理方式得出错误结论。
- 组织成本常被低估：术语认证、权限梳理、跨部门定义对齐是政治工作不是技术工作——"adopting a context layer ≈ upgrading the maturity of the data organization"（Pebblous），没有管理层承诺的团队应先做单场景试点再谈平台化。

第四部分 · 研判

第 14 章 评测的真伪：基准通胀与独立复现

14.1 基准地图：五个主流基准各测什么

基准	测什么	不测什么	2026 年公开水平
LoCoMo	多会话对话的单跳/多跳/开放域/时序问答（1540 题，10 段对话，平均 27.2 会话/588.2 轮）	任务执行类记忆；答案键本身有约 6.4% 错误（见 13.3）	头部宣称 90+（各家自评）
LongMemEval	超长历史中的精准召回（S 子集 500 题，平均 50.2 会话/11.5 万 token），专攻滑窗/简单向量失败的场景	工作流/机构知识	Mem0 宣称 94.4；独立复现 57.5（见 13.2）
BEAM	128K–10M token 尺度记忆（2000 题），专门防"全塞上下文"刷分	—	公开最高约 64，长程记忆仍是难题
ConvoMem / MemBench	ConvoMem 测对话记忆端到端；MemBench 共 26,637 条、11 类（含对抗干扰项 noisy 与推理密集 highlevel）	—	Mem0 在 ConvoMem 各类 30–45%（MemPal 引述口径）；MemBench highlevel 类 R@1 普遍最低（26–69%）
DMR / Personamem	DMR（Zep 引入）测深度记忆检索；Personamem 七类个性化任务（召回/最新偏好/偏好演化/成因/建议/推荐/泛化， arXiv:2602.14038 ）	DMR 为厂商引入基准；Personamem 覆盖个性化但不覆盖任务状态	DMR：Zep 自报 94.8 vs MemGPT 93.4

基准说明综合：[clawdbot2.in](#)、[maximem.ai](#)、[thesingularity.dev](#)。另注意：Mem0 四月算法的宣称增益集中在时序推理（+29.6）与多跳（+23.1）——恰好是前代纯向量方法最弱的题型。

14.2 宣称 vs 复现：2026 年最重要的一张对照表

把散落各处的同系统不同口径数字放进一张表，通胀的形状一目了然：

系统	基准	厂商/论文口径	独立或对手口径	差距
Mem0	LongMemEval	94.4（官方报告，2026.04，"Engineering Team"署名，无公开 harness 与日志）	57.5（Maximem 开源 harness）；49.0（arXiv:2603.04814，GPT-5-mini 三票判官）	–36.9 至 –45.4
Mem0	LoCoMo	92.5（同上官方报告）	66.9（memnode.dev 复现）	–25.6
Mem0	LoCoMo（自家论文）	单跳 J-score 67.13（2025 论文）	——与独立复现 66.9 高度吻合	自家论文反而接近复现值
Zep	LongMemEval-S	71.2（gpt-4o，2025 论文）	63.8（AgentOS 审计复现）	–7.4

Zep	LoCoMo	75.14 (Zep 自评, 并指控 Mem0 搞错说话人角色)	65.99 (Mem0 论文给 Zep 打分); 58.44 (Mem0 去对抗题重跑)	同系统 17 点横跨
MemGPT	DMR	93.4 (Zep 论文中引用)	——DMR 为 Zep 引入的基准	引入方自裁

汇总自：[genalphai.com](#)、[maximem.ai](#)、[arXiv:2603.04814](#)。所有第三方数字的 harness 配置同样未经审计。

这张表里最值得咀嚼的一行是第三行：**Mem0 自己 2025 年论文的单跳分数（67.13）与 2026 年独立复现（66.9）几乎重合，而官方博客宣称 92.5。**最合理的解释是宣称口径换了 harness 切片或判官配置（genalphai.com 的分析同此判断）——也就是说，inflation 不一定来自"造假"，更可能来自"换尺子"：换判官模型、去对抗题、调检索深度、换答案抽取 prompt，每一步都合法，加起来就是 25 个点。这也是为什么"看论文数字"比"看博客数字"相对可靠：论文通常保留完整方法学，营销材料不会。

14.3 测量工具本身在漏：基准的两处结构性缺陷

- **答案键错误**：Dell Zhang 2026 年 5 月的评述引 Penfield Labs 审计——LoCoMo 的 ground truth 在 1540 题中含 99 处计分错误，错误率约 6.4%；评述者自己注明 Penfield Labs 维护竞争性记忆项目、其在开审计仓复核后才转引（[maximem.ai 转述](#)）。6.4% 的噪声意味着：所有 LoCoMo 分数小数点后的差异都无意义，5 个点以内的差异难以与标注噪声区分。
- **判官漏洞**：同一审计给出，标准配置（gpt-4o-mini + 原始 prompt）的判官会接受 **62.81% 的"错误但沾边"的答案**——判官不是在判"对不对"，很大程度上在判"像不像对的"。这解释了为什么换判官模型分数剧烈波动，也解释了为什么"加严判官 + 三票制"（arXiv:2603.04814 的做法）会把分数整体拉低 20–40 点。

两个缺陷合起来给出一条实用阈值：**在 LoCoMo/LongMemEval 上，小于 10 个点的任何比较（跨系统或跨版本）都不构成证据**——它可能来自答案键噪声、判官抖动、检索深度差异的任意组合。

14.4 自建评测的最低配置（可操作）

综合各方建议与本书实测经验，一个可信的自测 harness 至少包含：

1. **从生产轨迹抽 50 条含矛盾的真实会话**（有"后来改口"的记忆，才有失效问题可测）；
2. **固定作答与评审模型**，且与被测系统默认抽取模型分开记录；判官加严：三票制或更严格的判分 prompt（参照 arXiv:2603.04814）；
3. **四类指标**：召回准确率（明确 R@k 还是端到端 QA，两者都报）、p50/p95 检索延迟、每查询注入 token 数、每条记忆的写入 LLM 成本；
4. **k 值诚实**：top-k 必须小于数据最小单元数（LoCoMo 场景 k<19），否则测的是重排器不是检索库（第 5.5 节）；
5. **消融开关**：记忆开/关对照（Braintrust 提醒：记忆可能帮倒忙）；混合检索开/关对照（验证静默降级）；
6. **报告方法学**：任何对外数字附 harness 配置。可直接用开源 harness 起步——Maximem 的 MIT 协议 harness（[maximem.ai](#)）可端到端跑 Mem0/Zep/SuperMemory 并自带"偏离公开协议之处"的说明文档。

14.5 被"记忆"一词包装过头的三个宣称

宣称	实际（源码实测证据）
"图谱记忆"（多家向量派系统）	Mem0 实体层是第二向量集合 + id 反向索引（main.py:638-643），无图遍历、无图查询语言；CrewAI 新版 EntityMemory 退化为 metadata 字符串列表（无图遍历，crewai 1.15.21 实测）
"AGM/记忆操作系统"	MemOS 的调度器实为记忆操作的消息队列与派发器，非 OS 进程调度（mem_scheduler 实测）；参数记忆标注"即将推出"；Dream 信号重启即失
"agentic memory 自主演化"	A-MEM 精简仓的链接扩展在主路径实际失效（序号≠UUID），"自主性"依赖未兑现的身份系统；evolution_history 从不 append

识别这类包装有一个通用判据：**宣称的名词与实现的抽象层是否同构**。"图谱"要求图遍历语义（BFS/最短路/子图匹配），反向索引不提供；"操作系统"要求资源隔离与调度语义，消息队列不提供；"自主"要求可靠的自主引用与可逆操作，序号引用与不可追溯改写不提供。名词借来的气势不会随实现到位——第 7 章的表示哲学问题（"真相放在什么表示里"）在这里反过来用：先问宣称的真相放在哪层抽象，再决定信多少。

- 在现有证据下：**2026 年记忆系统的真实分化在成本、延迟、部署边界与治理合规，不在检索精度**——后者被营销数字污染到不可用。
- 对外宣称必须区分三层：数据结构（真图还是反向索引）、机制（真时序失效还是 TTL）、效果（自报分数还是独立复现）。第一层读代码可验，第二层读实现可验，第三层只能自测。
- 论文数字 > 博客数字 > 落地页数字，可信度排序有方法学依据：论文保留完整 harness 描述，且经历过至少一次同行压力。
- 小于 10 个点的基准差异在现有测量工具精度下不构成证据。

边界与辩证

- "复现崩塌"的博客数据本身也未提供可复跑的 harness——对厂商与批评者都应保持同等怀疑；Maximem、Penfield Labs 均为竞争性记忆厂商，其"审计"有利益相关性（各方多有自述，值得肯定，但立场要计入）。
- 基准与真实负载的偏差是双向的：LongMemEval 高分不完全代表无用，只是不充分；任务执行类记忆（多步工作流的状态保持）目前根本没有公认基准。
- 自测也要防过拟合：50 条轨迹上调参后的结论不能外推为通用结论。
- 本章数字为 2026.09 快照，各系统迭代快，跨时间比较同样需要重跑。

第四部分 · 研判

第 15 章 工程选型：成本、边界与决策框架

15.1 成本结构：账单长在哪里

记忆层的成本不在"软件许可"，在三个随使用量增长的地方：**写入的 LLM 抽取调用**（每条消息 1 次以上）、**每轮检索注入的 token**、**存储与索引的持续增长**。第三方在 1 万月活、人均 20 轮/月、50 条记忆的负载下测算（2026 年 9 月定价页核对）：

方案	月成本	记忆模型	自托管路径	删除权实现
自建 Postgres + pgvector	~\$163–332	自己的行 + embedding	就是你的数据库	DELETE WHERE user_id=\$1
Mem0 平台	\$249 Pro 档（4 倍于其检索配额）	LLM 抽取事实，可选图	Apache-2.0（减平台优化）	按记忆/用户 API
Zep 云	\$375–750	双时态知识图谱	仅 Graphiti；自备图库	一次 API 删用户/线程/图
Letta 云	~\$1,020（不含 LLM token）	有状态记忆块与文件	Apache-2.0 自托管服务器	按 Agent；级联行为文档不明

测算与定价出处：[DEV Community \(含完整测算表\)](#)。托管层对标准负载约为自建基线的 2–6 倍。

还有定价模式的暗坑：Google Memory Bank 按 GiB 月计费（含修订版本）且父记忆无默认 TTL，"四月的历史配置决定十月的账单"——用量计费模式让账单随 Agent 采用率（而非人数）增长，试点便宜、生产昂贵。按操作/存储记忆/摄入字节计费的三种模式各有陷阱，签约前要把"什么在计费"问清。

token 侧的量级参考（厂商口径，看数量级即可）：典型客服 Agent 每次交互约 3150 输入 + 400 输出 token，1 万日活 × 人均 5 次/天 ≈ 月 2000–5000 美元 LLM 成本，记忆压缩类方案可省一半以上（[youmind.com](#)）；Mem0 四月算法自报约 6900 token/查询，SimpleMem 自报最高 30 倍 token 削减（[o-mega.ai](#)）。把这组数字与 14.1 的平台费相加，才是记忆层的全成本：**平台费 + 抽取 LLM 调用 + 注入 token + 存储**，四项里最容易被漏算的是抽取调用——它随对话量线性增长且用高质量模型计价。

15.2 一个务实的决策框架

综合源码实测与第三方分析，选型按四问排序：

决策问题	若答"是"	若答"否"
① 事实会频繁变更吗（用户档案、客户状态）？	时序图谱：Graphiti/Zep	向量派即可

③ 记忆需要人审计/合规审查吗？	文件派或文件为源的混合：Basic Memory、git 化 Letta	托管 API 亦可
③ Agent 生命周期内聚、模型能力强吗？	自编辑派：Letta / memory tool 模式	服务侧抽取：Mem0/LangMem 后台路径
③ 多跳关联是核心需求吗（组织知识、文档网络）？	图混合：Cognee/HippoRAG2 + 图库	稠密+稀疏混合即可

四问之外应补第五问（常被遗漏）：③ **记忆数据可以出域吗？**——医疗、金融、政务场景里"记忆存谁的云"是一票否决项：Anthropic memory tool（客户端文件、开发者持有）与自建栈是仅有的两个"数据不出域"选项，托管层全数出局（第 12.2 节）。

三个补充判断：**其一**，已在某框架/云内的团队先用自己的原生记忆（LangGraph Checkpointer+Store、CrewAI Memory、Vertex Memory Bank），**撞到天花板再换独立层**（o-mega.ai 建议，[链接](#)）；**其二**，最关键的边界问题是"记忆系统该拥有你应用的多少"——只做存储检索（pgvector 自建）还是接管抽取/消解/治理（托管全家桶），这决定迁移成本与锁定深度；**其三**，引入专门系统的正当理由是"能力"而非"精度"：双时态失效、自编辑块、实体消解——这些是真需要定制工程的能力（DEV Community）。"如果只是要 Agent 跨会话记住用户偏好，那不是买它们的理由。"

15.3 分阶段路线图：Day-1、Day-30、Day-90

选型框架回答"用谁"，路线图回答"什么时候做什么"。按投入产出排出的三阶段：

阶段	做什么	明确不做什么
Day-1 (MVP)	① 原始消息层（带时间戳全量留存，成本极低、事后补不了）；② namespace/scope 设计（user/agent/run 三键，学 Mem0 的输入防御三查）；③ 事实表加四字段：created_at / valid_at / access_count / durability；④ 相对时间锚定绝对日期	不建图、不建多路检索、不买托管层——此时检索纯向量都够用
Day-30 (成长)	① BM25/关键词信号补上（验证无静默降级）；② 衰减与驱逐按 durability 分流；③ 后台巩固任务（合并/摘要、带 dry_run）；④ 按第 14.4 节建 50 条轨迹的自测 harness，记忆开/关消融对照	不引入第二个记忆系统；不对对外报告任何基准数字
Day-90 (规模化)	① 用 harness 数据定位天花板：时序失效不够→评估 Graphiti；多跳不够→评估图派；治理不够→文件派混合；② 删除权合规管线（审计日志 + 派生物级联清理）；③ 多 Agent scope 权限模型	不因为"别人都在用 X"而迁移——迁移成本随记忆积累单调上升

这个路线图的核心逻辑：**所有"事后补不了"的决策（schema、scope、原文层）压到 Day-1，所有"随时可换"的决策（检索器、巩固策略、甚至整套记忆系统）推到 Day-30/90**。判断一件事属于哪类，问一句：如果三个月后推翻它，代价是改代码，还是迁移数据？

15.4 失效模式清单（生产环境的 Day-2 问题）

- **Demo-生产鸿沟**：演示可用因为整个对话塞得进一个窗口；生产崩溃因为用户跨会话、跨天、跨主题回来——记忆问题的定义性失败模式（Particula Tech）。
- **检索反噬**：过时事实、不相关细节、新增延迟会**降低**而非提升任务表现；没有受控前后对比就不知道记忆在帮忙还是帮倒忙（Braintrust 2026，转引自 [jakeinsight.com](#)）。
- **静默降级**：Mem0 向量后端不支持关键词检索时静默退化为纯语义（行 543 实测）——上线后要验证混合检索真的在跑。
- **最终一致性**：异步写路径下"写完立刻搜"可能搜不到；读屏障（CrewAI drain、Letta 重建提示）是对此的工程补偿。
- **嵌入维度漂移**：换嵌入模型后旧向量索引错配——HippoRAG 用 manifest 身份校验拒绝复用，CrewAI 抛 EmbeddingDimensionMismatchError；自己搭建时这两个防御都要补。
- **记忆投毒**：提示注入诱导 Agent 把恶意内容写成"用户约定/系统事实"，此后每轮检索都会召回它——这是比单次注入更持久的攻击面（第 1.6 节"安全假设"的具体化）。缓解：自主写入限额、敏感记忆双确认、写入来源审计；目前没有框架内置防护。
- **删除权合规**：GDPR/个保法视角下，"DELETE WHERE user_id=\$1"（自建）与逐条 API 删除（托管，Letta 的级联删除行为文档不明）成本完全不同（DEV Community 测算表）。
- **锁定与存续**：Assistants API 弃维（2026-08-26）证明厂商原语的存续周期不由你决定；记忆积累越久越值钱，迁移成本随时间上升——记忆的可移植性要在第一天设计（格式中立导出、命名空间规范）。

八条失效模式有个共同的预防原则：**让记忆系统的一切行为可观测**——写入有审计日志、降级有告警而非静默、巩固有 dry_run、删除有级联清单。Mem0 的 history 表、Cognee 的幂等标记、Basic Memory 的墓碑记录，都是在各自局部实现这条原则；自建系统应把它作为横切关注点一次做对。

研判

- 在现有证据下：**先自建朴素栈（pgvector + 命名空间 + 后台巩固任务），用真实轨迹度量出天花板后再引入专门记忆系统**，是期望成本最低的路径。
- 选型比较"表示与治理模型"（时间性/审计/生命周期归属）比比较"检索精度"更有决策价值（第 14 章已证精度数字不可信）。
- Day-1 的五个"补不了"决策（原文层、scope、四字段、时间锚定、审计）是整张路线图里杠杆最高的投入。

边界与辩证

- 自建同样有隐性成本：冲突消解、防抖、多租户隔离、审计日志——Mem0 用 3868 行核心代码换来的东西，自建者要自己写。
- "原生记忆够用"的前提是框架锁定可接受；Assistants API 前车之鉴说明原生≠稳妥。
- 本框架基于 2026 年 9 月的证据，定价与能力格局半年内可能显著变化。
- 成本测算表的负载假设（1 万 MAU、人均 50 条）未必匹配你的业务，量级结论可参考，绝对值需重算。
- "记忆投毒"的缓解手段是本书基于攻防常识的工程推断，非实测结论；该方向的公开研究仍在早期。

第四部分 · 研判

第 16 章 开放问题与前瞻研判

16.1 八个未解的根本问题

- 长程记忆的基准荒漠**：LoCoMo/LongMemEval 测对话问答，BEAM 测超长召回，但"多步工作流的状态保持""机构知识的沉淀质量"没有公认基准——这个领域连"什么算记住了"都没共识。
- 遗忘的正确性**：删除是不可逆操作，而"该忘什么"的判断由 LLM 做，错误无回滚机制。Generative Agents 的 filling 证据链是目前唯一的可追溯设计，但只存在于仿真里；视图级遗忘（OpenHands）是工程上最安全的折中。
- 巩固的误差雪球**：insight 写回后进入检索池，错误抽象会被后续巩固引用放大。目前没有任何系统对巩固产物做独立的置信度衰减（Cognee 的反馈权重与 A-MEM 的演化都只增不减）。
- 表示的不可迁移**：Mem0 的命名空间、Graphiti 的双时态边、Letta 的块、Basic Memory 的 wikilink——没有共同交换格式，迁移等于重建。三平台的记忆导入导出（最多 5GB ZIP）只是聊天历史的搬运，不是语义记忆的迁移。
- 治理黑盒**：Dreaming 类后台改写对用户不可审计；EU AI Act 透明度要求（2026 年 8 月全面生效）与之的冲突尚未有产品级答案。
- 写入吞吐与时序完整性的对立**：Graphiti 要求顺序写入保证失效链正确，高频对话场景却要求并行高吞吐——目前没有两全的公开方案，这是时序派落地的最大工程障碍。
- 记忆的归属与延续（宪制维度）**：2026 年"宪制记忆架构"论文（[arXiv:2603.04740](https://arxiv.org/abs/2603.04740)）把两个被全体系统回避的问题正式化——**记忆归谁**（现在全部默认归用户/会话，Agent 自身的经验积累无处安放）与**实例延续**（实例终止后记忆如何继承：多个前代实例时继承优先级是什么、新实例"理解"继承记忆如何验证）。这两个问题在"Agent 是工具"时可以回避，在"Agent 是长期数字实体"时无法回避。
- 自主改写的安全边界**：A-MEM 的五个陷阱（第 11.3 节）只是工程实现层的失误；更根本的问题是——允许 LLM 改写自身记忆的安全边界在哪里（哪些字段可改、改动如何被否决、被注入的改写如何识别），目前没有任何系统给出形式化答案。

16.2 阶段性研判（2026–2030）

方向	研判	信心与依据
记忆与窗口边界	长窗口+缓存持续压缩外部记忆的必要性，但时序失效与治理需求保住外部层的存在	中高：Anthropic token 数据、BEAM 仍仅 64 分
托管化	四家托管层已收敛，记忆成为云厂商标配基础件；开源内核保持架构多样性	高：2026 年中发布潮

效率轴竞争	"每点 F1 花多少 token"取代"绝对 F1"成为主竞争面；压缩/调度派（SimpleMem、MemTree、MemoryOS）机制会被托管层吸收	中高：SimpleMem 30× 数字 + 四托管层同架构收敛 + ICLR MemAgents workshop 设立
文件派的地位	作为"可审计层"长期存在，与机器检索层组成双形态，不会消失	高：AGENTS.md 6 万项目、Claude Code 实践、Basic Memory 生态补位
架构内记忆	参数记忆/KV-Cache 记忆 3–5 年内仍处研究期，不构成对②派的替代	中：MemOS para_mem"即将推出"状态持续，无生产案例
记忆标准	行业会出现记忆交换格式/治理接口的标准化尝试（MCP memory server 已是雏形）	中：无公开路线图，仅趋势推断
评估	独立第三方 harness 将成为采购前置条件——起点已出现（Maximem 开源 MIT harness、arXiv:2603.04814 的三票判官协议），但尚未被行业广泛采用	中：已有多方指出且开源实现已存在，掣肘在厂商配合度
监管驱动	记忆治理（可见/可改/可删/审计）将从合规负担转化为产品差异化能力，先行者受益	中：EU AI Act 已生效、消费平台已上线记忆面板与导出

16.3 三层读者各自能带走什么

读者	能直接用的	需要二次开发的
使用者/产品经理	第 12 章产品对比选消费工具；第 14 章的怀疑清单看穿营销数字（<10 点差异不构成证据）；治理五项检查单（可见/可改/可删/导出/审计）作为采购必问项	—
开发者	第 15 章五问框架与三阶段路线图；第 3 章 scope 设计；第 5-6 章的实测公式（RRF/半衰期/EMA 反馈）直接可抄；Day-1 五决策清单	自建读屏障、索引身份校验、删除权合规；harness（第 14.4 节）；记忆投毒防护
研究者	第 7 章能力矩阵找空白点（溯源+巩固是普遍短板）；第 16 章八个开放问题即选题清单；第 14 章的测量缺陷分析（答案键噪声 6.4%、判官 62.81% 误收）是设计新基准的起点	复现论文结果前先核对开源精简仓与实验仓的差异（A-MEM 案例）；跨 harness 数字不可直接比较

研判

- 记忆问题的终局不是"更大的窗口"，而是窗口内策展与窗口外治理的分工：窗口负责当下的连贯，外部系统负责时间性与问责。
- 对工程团队：现在最值得投入的不是追新框架，而是打好三个地基——命名空间设计、事实级时间戳、可审计的写路径。三者无论最终选谁都需要。
- 领域下一个最大的确定性变量是评估基建：谁先让"可信的自测"变便宜，谁就改变采购行为。

边界与辩证

- 上述研判全部为阶段性判断：如果模型原生支持持久记忆（架构级改动），外部记忆层的价值模型需要重写。
- 标准化推断的依据较弱（MCP memory server 只是参考实现之一），可能被平台锁定竞争扼杀。
- "三个地基"建议对超小团队可能过重——10 用户内部工具用框架自带 memory 就够，不必过度设计。
- 效率轴研判依赖 SimpleMem 等论文自报数字，若其方法学经不起复现，竞争轴转移的时间表会推后。

最后回到第一性原理：Agent 记忆的全部工程，是在"有限的、易失的、计费的窗口"与"无限的、持久的、会腐烂的历史"之间，做一系列可辩护的取舍。凡是宣称"彻底解决"的方案，都值得先看它的失效模式清单——本书各章的「边界与辩证」栏，就是为此而写。

附录

附录 A 术语表

术语	定义
双时态 (bitemporal)	同时记录事实的有效时间（何时为真）与系统认知时间（何时得知）两个时间轴

巩固（consolidation）	对已存记忆的后置加工：合并、抽象（反思）、纠错，源于认知科学的记忆巩固理论
PPR（Personalized PageRank）	从指定种子节点出发的随机游走打分，用于图上扩散检索信号
RRF（Reciprocal Rank Fusion）	按倒数排名融合多路检索结果的公式，避免不同分布分值直接相加
MMR（Maximal Marginal Relevance）	兼顾与查询相关性及结果间多样性的重排准则，用于去冗余
Cross-encoder	把查询与候选拼在一起过同一模型的精排器，精度高于双塔向量但成本高
OpenIE	开放信息抽取：从文本抽（主语,谓语,宾语）三元组，不预设关系类型
Namespace / Scope	记忆的多租户分片键（user_id/agent_id/run_id 等），检索过滤的强制边界
MCP（Model Context Protocol）	Anthropic 2024 年 11 月发布的开放协议，标准化 Agent 与外部数据源/工具的连接；2025 年 12 月捐赠给 Linux 基金会旗下 Agentic AI Foundation，现为准事实标准。它搬运上下文，不生产上下文
企业上下文（Context Layer）	组织级、被治理、供所有 Agent 与应用共享读取的背景信息层，覆盖语义定义、业务规则、权限映射与来源溯源；由"人工治理 + AI 提议"维护，区别于 Agent 私有的经验性记忆（见第 13 章）
Context Repo	上下文层的可版本化发布单元：开放格式、可导出、经 MCP 跨模型共享，一处修正全体接入模型受益（第 13.3 节）
读屏障（read barrier）	读操作前强制等待/排空异步写队列，保证读必见已提交写的机制
视图级遗忘	不删数据、仅把内容从喂给 LLM 的窗口视图移除的遗忘方式（可回溯）
MCP（Model Context Protocol）	Anthropic 发起的模型-工具连接协议，其参考实现含基于知识图谱的 memory server
基准通胀（benchmark inflation）	厂商在自选配置下报告的高分在独立测试环境中显著缩水现象
Context Rot（上下文腐烂）	随上下文变长，模型对关键信息的利用效率下降的现象
Compaction / Condenser	上下文将满时把旧内容折叠为摘要的机制（Anthropic/OpenHands 各自的实现名）
Zettelkasten	卡片盒笔记法：原子化笔记 + 双向链接 + 持续改写；A-MEM 的设计原型
海马体索引理论	神经科学理论：少量线索经海马体扩散激活皮层记忆存储；HippoRAG 的机制来源
Provenance（溯源）	记忆条目指回其来源证据的能力（filling/sources/fact_source_counts 等实现）
半衰期衰减	$decay = 0.5^{(age_days/30)}$ 式的时间降权公式：每过半衰期权重减半
静默降级	组件能力缺失时不报错、退化为更弱模式的行为（如混合检索退化为纯语义）
Dreaming	离线巩固的产品化命名：后台跨会话合成、修订时效性记忆（MemOS Dream 插件 / ChatGPT Dreaming）
MemCube	MemOS 的统一记忆单元抽象：把明文/激活/参数三种形态的记忆封装为带来源、版本、使用历史的可调度对象
三范式（组织策略）	2025 年综述对记忆系统的划分：平面检索型 / 显式结构型 / 策略管理型（按组织与调度策略而非表示划分）
LLM 判官（judge）	用 LLM 对被测系统答案打分的评测组件；其模型与 prompt 的选择可造成数十分数的分数波动
答案键（ground truth）噪声	基准数据集自带标注的错误率（如 LoCoMo 约 6.4%），构成所有分数的下限噪声
证据缺口追踪（evidence-gap tracking）	闭环检索中显式维护"已验证证据/缺失信息"清单、缺口闭合即停止检索的机制（MemR3）
CQRS（文件派架构）	写侧权威（文件）与读侧投影（索引库）分离的模式：投影可随时重建，权威数据零迁移
记忆投毒	通过提示注入把恶意内容写入持久记忆、使其被持续检索利用的攻击面

附录 B 主要参考文献与数据来源

源码逐行实测（版本以正文标注为准，路径与行号可复核）：

- Mem0 — github.com/mem0ai/mem0 (v2.0.20)
- Graphiti — github.com/getzep/graphiti (0.30.2)
- Letta (MemGPT V1 归档) — github.com/letta-ai/letta (0.16.8)；活跃仓 github.com/letta-ai/letta-code
- LangMem — github.com/langchain-ai/langmem (0.0.30)
- MemOS — github.com/MemTensor/MemOS (2.0.33)；官网 memos.openmem.net
- CrewAI — github.com/crewAIInc/crewAI (1.15.21)
- OpenHands — github.com/OpenHands/OpenHands (1.18.0 前端仓；SDK 在 [software-agent-sdk](https://github.com/software-agent-sdk))
- HippoRAG — github.com/OSU-NLP-Group/HippoRAG；论文 arxiv.org/abs/2502.14802 (v2)、arxiv.org/abs/2405.14831 (v1)
- Cognee — github.com/topoteretes/cognee
- Basic Memory — github.com/basicmachines-co/basic-memory
- A-MEM — github.com/agiresearch/A-mem；论文 arxiv.org/abs/2502.12110；实验仓 [WujiangXu/AgenticMemory](https://github.com/WujiangXu/AgenticMemory)
- Generative Agents — github.com/joonspk-research/generative_agents

论文：

- Zep: A Temporal Knowledge Graph Architecture — arxiv.org/abs/2501.13956
- MemGPT: Towards LLMs as Operating Systems — arxiv.org/abs/2310.08560
- Generative Agents: Interactive Simulacra — arxiv.org/abs/2304.03442
- MemOS 论文 — arxiv.org/abs/2507.03724
- Agent Memory 综述与横评（MemTree/MemoryOS 等） — arxiv.org/pdf/2504.19413、arxiv.org/pdf/2505.22101
- Personamem / 自适应记忆结构（含 MemR3、O-Mem 等基线描述） — arxiv.org/abs/2602.14038
- A Constitutional Memory Architecture（隐含假设与治理/延续/身份维度） — arxiv.org/abs/2603.04740
- Mem0 独立复现（三票判官协议） — arxiv.org/abs/2603.04814
- Evermind EverOS 评测 — arxiv.org/abs/2601.02163

2026 年行业动态与评测分析（2026-09-26 访问）：

- Particula Tech: Mem0 vs Zep vs Letta vs Cognee — particula.tech/blog
- DEV Community: AI Agent Memory Tiers Explained — dev.to/saaswithalex
- DEV Community: Mem0 vs Zep vs Letta: Production Cost — dev.to/jangwook_kim
- Jake's Insights: AI Agent Memory Tools Compared — jakeinsight.com
- Clawd Bot: Complete 2026 Guide — clawdbot2.in
- Taskade: The History of Agent Memory — taskade.com/blog/agent-memory-history
- o-mega: AI Agent Memory in 2026 / Self-Improving AI Agents — o-mega.ai、o-mega.ai/articles/self-improving-ai-agents-the-2026-guide
- Maximem: Best AI Agent Memory Platforms 2026（含开源 MIT harness） — maximem.ai
- Gen α: 4 Launches in 7 Weeks（宣称 vs 复现对照） — genalphai.com
- thesingularity.dev: Take a Look, It's in a Book（检索口径分解） — thesingularity.dev
- YouMind: MemOS Deep Dive（含 token 成本测算） — youmind.com
- CrabTalk: a survey of persistent memory — crabtalk.ai
- ChatForest: AI Agent Memory Patterns — chatforest.com
- MindPattern: 冲突基准声称观察 — mindpattern.ai/f/18020
- signals.tw（繁中）：agent memory 是什麼 — signals.tw

延伸：Anthropic context engineering 指南与 memory tool 文档见 docs.anthropic.com；MCP 参考实现见 github.com/modelcontextprotocol。第三方博客观点均标注于正文，其方法论细节（harness 配置）未经独立审计，引用时请注意。

《Agent Memory 绿皮书》v3.1（增补企业上下文章节）· 2026-09-26 · 本绿皮书全部源码级结论来自开源仓库逐行实测，第三方观点与数据标注出处链接，引用请注明署名与版本。